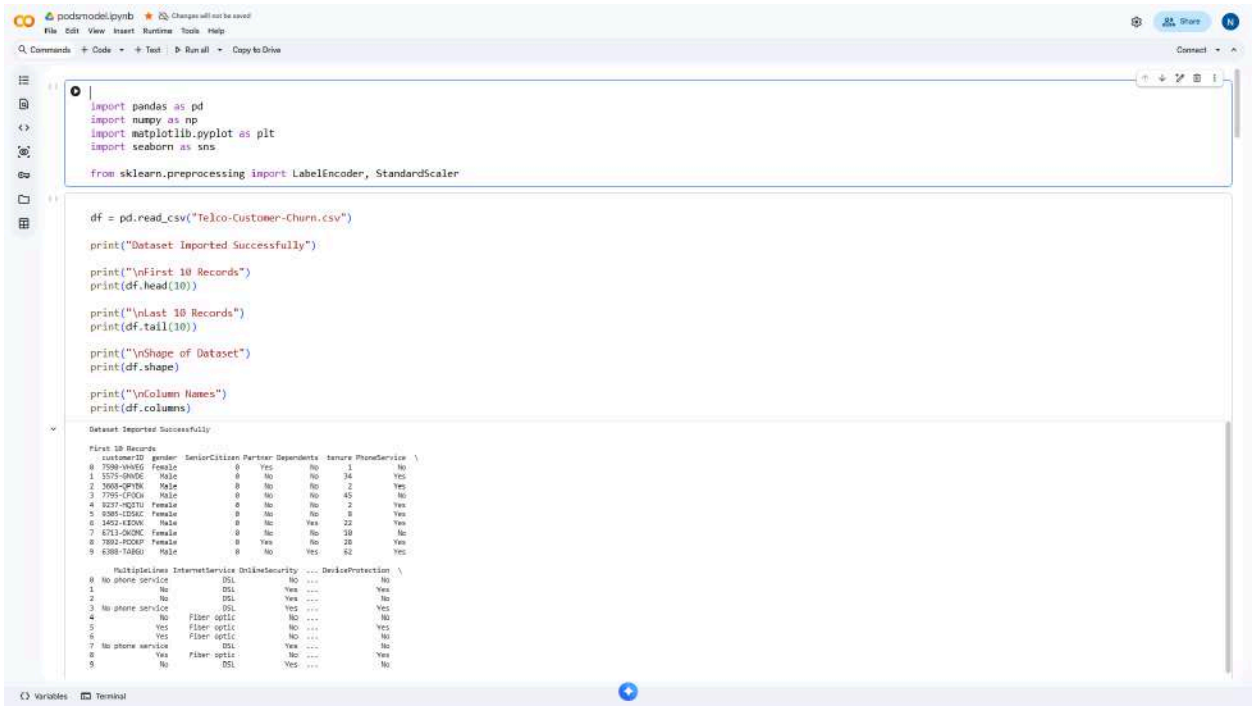
Name:Nithin S
Roll no:241801191

Ex1: Data Preprocessing and Cleaning
Problem Statement:

A telecommunications company is experiencing customer churn (customers leaving the service). Before building a machine learning model to predict churn, the data science team must clean and preprocess the customer data to ensure high-quality input for analysis.
You are a Data Scientist working for the company. Your task is to preprocess the customer dataset by handling missing values, removing inconsistencies, transforming categorical features, scaling numerical attributes, and visualizing important relationships.
Learning Objective:

The objective of this experiment is to enable students to understand and perform the essential steps involved in data preprocessing. This experiment will provide a strong foundation for preparing high-quality data for subsequent machine learning and predictive analytics tasks.
Tasks to Perform:

1. Import the required Python libraries.
2. Load the Telco Customer Churn Dataset into a Pandas DataFrame. And display First 10 records, Last 10 records, Shape of the dataset, Column names.
3. Explore the Dataset and display Dataset information, Summary statistics, Number of missing values, Number of duplicate records
4. Clean the data
a. Remove duplicate customer records.
b. Replace missing values
i. Numerical columns → Median
ii. Categorical columns → Mode
c. Verify that no missing values remain.
5. Encode categorical columns: Gender, Partner, Dependents, Contract, PaymentMethod, InternetService, Churn. Use appropriate encoding techniques. 6. Standardise the numerical features: Tenure, MonthlyCharges, TotalCharges, using StandardScaler.
7. Plot Distribution of MonthlyCharges, Distribution of Tenure.
8. Create a Count Plot showing Contract vs Churn.
9. Draw a Box Plot of MonthlyCharges grouped by Churn.
10. Display the Correlation Heatmap of all numerical features.


podsmodel.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all Copy to Drive
Connect
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
df = pd.read_csv("Telco-Customer-Churn.csv")
print("Dataset Imported Successfully")
print("\nFirst 10 Records")
print(df.head(10))
print("\nLast 10 Records")
print(df.tail(10))
print("\nShape of Dataset")
print(df.shape)
print("\nColumn Names")
print(df.columns)
Dataset Imported Successfully
First 10 Records
customerID gender SeniorCitizen Partner Dependents tenure PhoneService \
0 7590-VHVEG Female 0 Yes No 1 No
1 5575-GNVDE Male 0 No No 34 Yes
2 3668-QPYBK Male 0 No No 2 Yes
3 7795-CFOCW Male 0 No No 45 No
4 9237-HQITU Female 0 No No 2 Yes
5 9305-CDSKC Female 0 No No 8 Yes
6 1452-KIOVK Male 0 No Yes 22 Yes
7 6713-OKOMC Female 0 No No 10 No
8 7892-POOKP Female 0 Yes No 28 Yes
9 6388-TABGU Male 0 No Yes 62 Yes
MultipleLines InternetService OnlineSecurity ... DeviceProtection \
0 No phone service DSL No ... No
1 No DSL Yes ... Yes
2 No DSL Yes ... No
3 No phone service DSL Yes ... Yes
4 No Fiber optic No ... No
5 Yes Fiber optic No ... Yes
6 Yes Fiber optic No ... No
7 No phone service DSL Yes ... No
8 Yes Fiber optic No ... Yes
9 No DSL Yes ... No
Variables Terminal

podsmodel.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Copy to Drive

Connect

```
#Within 5 [241801191]
print("\nDataset Information")
print(df.info())

print("\nSummary Statistics")
print(df.describe())

print("\nMissing Values")
print(df.isnull().sum())

print("\nDuplicate Records")
print(df.duplicated().sum())
```

```
Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   object 
 1   gender            7043 non-null   object 
 2   SeniorCitizen     7043 non-null   int64  
 3   Partner           7043 non-null   object 
 4   Dependents        7043 non-null   object 
 5   tenure            7043 non-null   int64  
 6   PhoneService      7043 non-null   object 
 7   MultipleLines     7043 non-null   object 
 8   InternetService   7043 non-null   object 
 9   OnlineSecurity    7043 non-null   object 
 10  OnlineBackup      7043 non-null   object 
 11  DeviceProtection  7043 non-null   object 
 12  TechSupport       7043 non-null   object 
 13  StreamingTV       7043 non-null   object 
 14  StreamingMovies   7043 non-null   object 
 15  Contract          7043 non-null   object 
 16  PaperlessBilling  7043 non-null   object 
 17  PaymentMethod     7043 non-null   object 
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object 
 20  Churn             7043 non-null   object 
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None

Summary Statistics
       SeniorCitizen       tenure  MonthlyCharges
count    7043.000000  7043.000000     7043.000000
mean        0.162147    32.371149       64.761692
std         0.368612    24.559481       30.090047
min         0.000000     0.000000       18.250000
25%         0.000000     9.000000       35.500000
50%         0.000000    29.000000       70.350000
75%         0.000000    55.000000       89.850000
max         1.000000    72.000000      118.750000

Missing Values
customerID          0
gender              0
```

Variables Terminal

podsmodel.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Copy to Drive

Connect

```
#Experiment -2
#Within 5 [241801191]
#step 4a
print("Duplicate Records Before:", df.duplicated().sum())

df.drop_duplicates(inplace=True)

print("Duplicate Records After:", df.duplicated().sum())
```

```
Duplicate Records Before: 0
Duplicate Records After: 0
```

```
#Within 5 [241801191]
numerical_columns = df.select_dtypes(include=["int64","float64"]).columns

for col in numerical_columns:
    df[col].fillna(df[col].median(), inplace=True)

    #step 4b
    categorical_columns = df.select_dtypes(include=["object"]).columns

for col in categorical_columns:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

```
/tmp/ipykernel_1735/3401636176.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df[col].fillna(df[col].median(), inplace=True)
/tmp/ipykernel_1735/3401636176.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df[col].fillna(df[col].median(), inplace=True)
/tmp/ipykernel_1735/3401636176.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

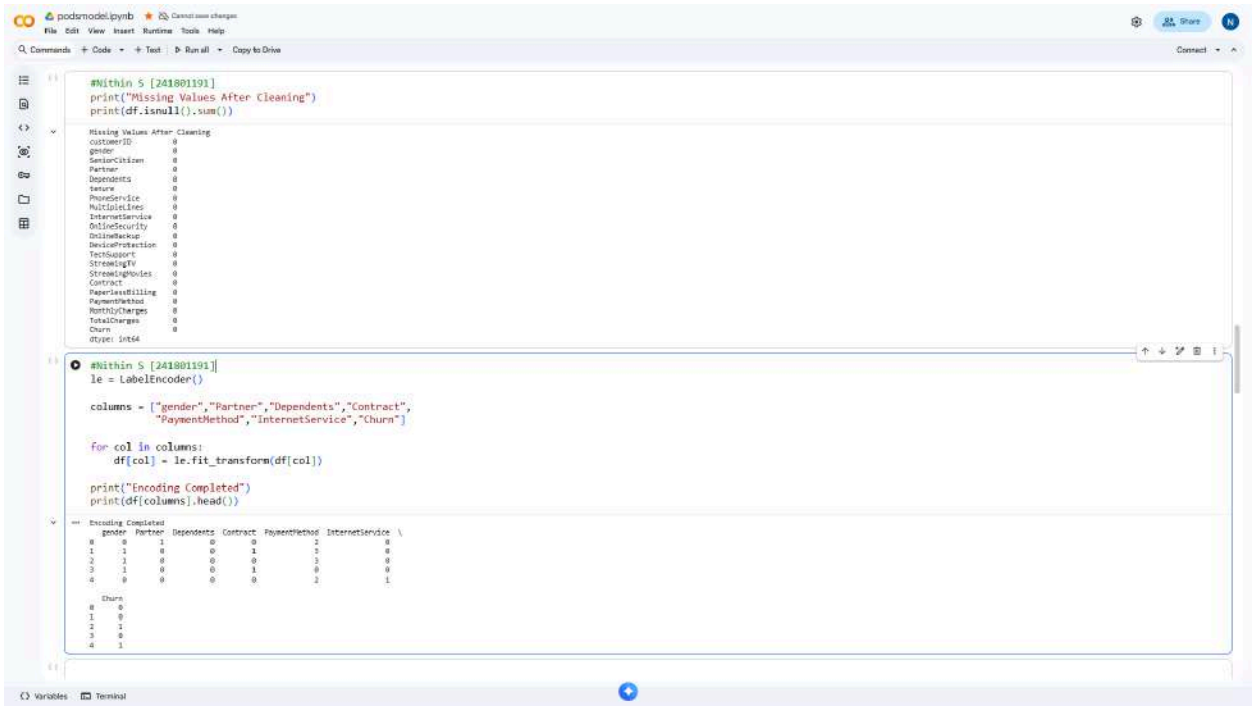
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df[col].fillna(df[col].median(), inplace=True)
/tmp/ipykernel_1735/3401636176.py:13: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df[col].fillna(df[col].mode()[0], inplace=True)
```

Variables Terminal


podsmodel.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all Copy to Drive
Connect
#Within S [241801191]
print("Missing Values After Cleaning")
print(df.isnull().sum())
Missing Values After Cleaning
customerID 0
gender 0
SeniorCitizen 0
Partner 0
Dependents 0
tenure 0
PhoneService 0
MultipleLines 0
InternetService 0
OnlineSecurity 0
OnlineBackup 0
DeviceProtection 0
TechSupport 0
StreamingTV 0
StreamingMovies 0
Contract 0
PaperlessBilling 0
PaymentMethod 0
MonthlyCharges 0
TotalCharges 0
Churn 0
dtype: int64
#Within S [241801191]
le = LabelEncoder()
columns = ["gender","Partner","Dependents","Contract",
"PaymentMethod","InternetService","Churn"]
for col in columns:
df[col] = le.fit_transform(df[col])
print("Encoding Completed")
print(df[columns].head())
Encoding Completed
gender Partner Dependents Contract PaymentMethod InternetService \
0 0 1 0 0 2 0
1 1 0 0 1 3 0
2 1 0 0 0 3 0
3 1 0 0 1 0 0
4 0 0 0 0 2 1
Churn
0 0
1 0
2 1
3 0
4 1
Variables Terminal

```
#Within 5 [241001191]
scaler = StandardScaler()

cols = ["tenure","MonthlyCharges","TotalCharges"]

# Convert 'TotalCharges' to numeric, coercing errors to NaN
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
# Fill any NaN values that resulted from coercion with the median
df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)

df[cols] = scaler.fit_transform(df[cols])

print(df[cols].head())
```

```
     tenure  MonthlyCharges  TotalCharges
0 -1.277445       -1.160323     -0.994242
1 -0.066327       -0.259629     -0.173244
2 -1.236724       -0.362660     -0.959674
3 -0.514251       -0.746535     -0.195766
4 -1.236724        0.197365     -0.940470
/tmp/ipykernel_5710/1914059551.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

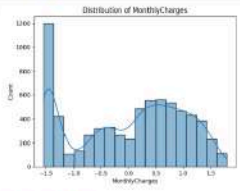
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)
```

```
#Within 5 [241001191]
sns.histplot(df["MonthlyCharges"], kde=True)

plt.title("Distribution of MonthlyCharges")

plt.show()
```

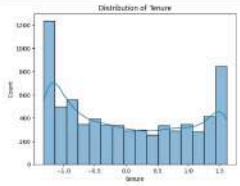



```
#Within 5 [241001191]
sns.histplot(df["tenure"], kde=True)

plt.title("Distribution of Tenure")

plt.show()
```

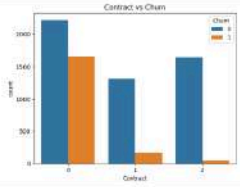





```
#Within 5 [241001191]
sns.countplot(x="Contract", hue="Churn", data=df)

plt.title("Contract vs Churn")

plt.show()
```




```
#Within 5 [241001191]
sns.boxplot(x="Churn", y="MonthlyCharges", data=df)

plt.title("Monthly Charges by Churn")

plt.show()
```

Ex2: Data Discovery and Preparation

Learning Objective:

The objective of this experiment is to perform data discovery and exploratory data analysis on a real-world dataset.

Question:

A telecommunications company has observed an increase in customer churn, leading to revenue loss. The management wants to understand customer behavior and identify patterns before developing a machine learning model to predict churn.

As a Data Analyst, your responsibility is to discover meaningful insights from the dataset, identify data quality issues, perform exploratory data analysis, detect outliers, and prepare the dataset for predictive analytics.

Tasks to Perform:

1. Reuse Ex:1 Steps 1- 6, 10.
2. Analyze the distribution of: Customer Churn, Gender, Contract Type, Payment Method. 3. Visualize the distribution of: MonthlyCharges, TotalCharges Tenure using Histograms and Boxplots.
4. Detect outliers in numerical features using the Interquartile Range (IQR) method. 5. Create suitable visualizations (Count Plot, Box Plot, Histogram, or Scatter Plot) to study the relationship between: Contract Type and Churn, Internet Service and Churn, Monthly Charges and Churn, Tenure and Churn.
6. Do the below analytical questions:

• Which contract type has the highest customer churn?
• Which payment method is most frequently used by customers who churn? • Do customers with shorter tenure tend to churn more often?
• Which customer group has the highest monthly charges?
• Which Internet Service category experiences the highest churn?
• Is there any relationship between monthly charges and churn?
• Which features appear to be the strongest indicators of customer churn? 7. Split the cleaned dataset into Training (80%) and Testing (20%) datasets. 8. Save the final processed dataset as a new CSV file
named CustomerChurn_Preprocessed.csv.

```python
#Within S - 241801191
#EX-2
#step 1

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
```

```python
#Within S - 241801191
#EX-2
#step 2

df = pd.read_csv("Titanic-Dataset.csv")

df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

```python
#Within S - 241801191
#EX-2
#step 3

print(df.shape)

df.info()

df.describe()
```

```python
df.info()

df.describe()
```

```
(891, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```python
#Within S - 241801191
#EX-2
#step 4

imputer = SimpleImputer(strategy='mean')

df['Age'] = imputer.fit_transform(df[['Age']])
```

75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
#Nithin S - 241801191
#EX-2
#step 4

imputer = SimpleImputer(strategy='mean')

df['Age'] = imputer.fit_transform(df[['Age']])
```

```
#Nithin S - 241801191
#EX-2
#step 5

df['Cabin'] = df['Cabin'].fillna("Unknown")
```

```
#Nithin S - 241801191
#EX-2
#step 6

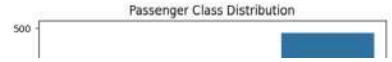
mode_value = df['Embarked'].mode()[0]

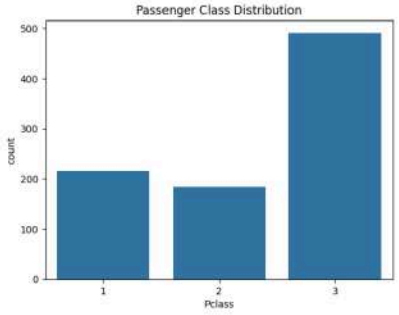
df['Embarked'] = df['Embarked'].fillna(mode_value)
```

+ Code + Text

```
#Nithin S - 241801191
#EX-2
#step 7

sns.countplot(x='Pclass', data=df)
plt.title("Passenger Class Distribution")
plt.show()
```






```
#Nithin S - 241801191
#EX-2
#step 8

female_survived = df[(df['Sex']=="female") & (df['Survived']==1)]

print(female_survived['Name'])
```

```
1      Cumings, Mrs. John Bradley (Florence Briggs Th...
2                                  Heikkinen, Miss. Laina
3           Futrelle, Mrs. Jacques Heath (Lily May Peel)
8       Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9                     Nasser, Mrs. Nicholas (Adele Achem)
                              ...
874                Abelson, Mrs. Samuel (Hannah Wizosky)
875                     Najib, Miss. Adele Kiamie "Jane"
879         Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
880          Shelley, Mrs. William (Imanita Parrish Hall)
887                        Graham, Miss. Margaret Edith
Name: Name, Length: 233, dtype: object
```

```
#Nithin S - 241801191
#EX-2
```

```python
#Nithin S - 241801191
#EX-2
#step 9

result = df[(df['Pclass']==3) & (df['Age']<18)]

print(result)
```

```
     PassengerId  Survived  Pclass                                     Name  \
7              8         0       3              Palsson, Master. Gosta Leonard
10            11         1       3          Sandstrom, Miss. Marguerite Rut
14            15         0       3      Vestrom, Miss. Hulda Amanda Adolfina
16            17         0       3                      Rice, Master. Eugene
22            23         1       3              McGowan, Miss. Anna "Annie"
..           ...       ...     ...                                     ...
844          845         0       3                      Culumovic, Mr. Jeso
850          851         0       3  Andersson, Master. Sigvard Harald Elias
852          853         0       3                   Boulos, Miss. Nourelain
869          870         1       3           Johnson, Master. Harold Theodor
875          876         1       3          Najib, Miss. Adele Kiamie "Jane"

        Sex   Age  SibSp  Parch   Ticket     Fare    Cabin Embarked
7      male   2.0      3      1   349909  21.0750  Unknown        S
10   female   4.0      1      1  PP 9549  16.7000       G6        S
14   female  14.0      0      0   350406   7.8542  Unknown        S
16     male   2.0      4      1   382652  29.1250  Unknown        Q
22   female  15.0      0      0   330923   8.0292  Unknown        Q
..      ...   ...    ...    ...      ...      ...      ...      ...
844    male  17.0      0      0   315090   8.6625  Unknown        S
850    male   4.0      4      2   347082  31.2750  Unknown        S
852  Female   9.0      1      1     2678  15.2458  Unknown        C
869    male   4.0      1      1   347742  11.1333  Unknown        S
875  female  15.0      0      0     2667   7.2250  Unknown        C

[78 rows x 12 columns]
```

```python
#Nithin S - 241801191
#EX-2
#step 10

result = df[(df['Pclass']==1) & (df['Age']>40)]

print(result)
```

```
    PassengerId  Survived  Pclass  \
6             7         0       1
11           12         1       1
35           36         0       1
52           53         1       1
54           55         0       1
```

```
#Nithin S - 241801191
#EX-2
#step 11

result = df[(df['Pclass']==1) &
            (df['Age']>40) &
            (df['Survived']==1)]

print(result)
```

```
     PassengerId  Survived  Pclass  \
11            12         1       1
52            53         1       1
187          188         1       1
194          195         1       1
195          196         1       1
268          269         1       1
275          276         1       1
299          300         1       1
337          338         1       1
366          367         1       1
380          381         1       1
449          450         1       1
453          454         1       1
460          461         1       1
496          497         1       1
513          514         1       1
523          524         1       1
556          557         1       1
571          572         1       1
587          588         1       1
591          592         1       1
599          600         1       1
621          622         1       1
630          631         1       1
645          646         1       1
647          648         1       1
660          661         1       1
707          708         1       1
712          713         1       1
765          766         1       1
779          780         1       1
796          797         1       1
820          821         1       1
829          830         1       1
856          857         1       1
857          858         1       1
862          863         1       1
871          872         1       1
879          880         1       1

                                                  Name     Sex   Age  SibSp  \
```

```
#Nithin S - 241801191
#EX-2
#step 12

result = df[(df['Sex']=="male") &
            (df['Fare']>100)]

print(result[['Name','Fare']])
```

```
                                        Name      Fare
27                  Fortune, Mr. Charles Alexander  263.0000
118                     Baxter, Mr. Quigg Edmond  247.5208
305                 Allison, Master. Hudson Trevor  151.5500
332                      Graham, Mr. George Edward  153.4625
373                            Ringhini, Mr. Sante  135.6333
377                     Widener, Mr. Harry Elkins  211.5000
390                   Carter, Mr. William Ernest  120.0000
438                           Fortune, Mr. Mark  263.0000
505  Penasco y Castellana, Mr. Victor de Satode  108.9000
527                           Farthing, Mr. John  221.7792
544                    Douglas, Mr. Walter Donald  106.4250
550                  Thayer, Mr. John Borland Jr  110.8833
557                           Robbins, Mr. Victor  227.5250
659                    Newell, Mr. Arthur Webster  113.2750
660                  Frauenthal, Dr. Henry William  133.6500
679            Cardeza, Mr. Thomas Drake Martinez  512.3292
698                    Thayer, Mr. John Borland  110.8833
737                      Lesurer, Mr. Gustave J  512.3292
802          Carter, Master. William Thornton II  120.0000
```

```
#Nithin S - 241801191
#EX-2
#step 13

result = df[(df['Embarked']=="C") &
            (df['Pclass']==2)]

print(result)
```

```
     PassengerId  Survived  Pclass  \
9             10         1       2
43            44         1       2
122          123         0       2
135          136         0       2
181          182         0       2
292          293         0       2
308          309         0       2
361          362         0       2
389          390         1       2
```

```python
#Nithin S - 241801191
#EX-2
#step 14

result = df[df['SibSp']>2]

print(result)
```

```
686                      Panula, Mr. Jaako Arnold    male  14.000000
726       Renouf, Mrs. Peter Henry (Lillian Jefferys)  female  30.000000
787                        Rice, Master. George Hugh    male   8.000000
792                          Sage, Miss. Stella Anna  female  29.699118
813               Andersson, Miss. Ebba Iris Alfrida  female   6.000000
819                     Skoog, Master. Karl Thorsten    male  10.000000
824                      Panula, Master. Urho Abraham    male   2.000000
846                          Sage, Mr. Douglas Bullen    male  29.699118
850             Andersson, Master. Sigvard Harald Elias    male   4.000000
863                    Sage, Miss. Dorothy Edith "Dolly"  female  29.699118

     SibSp  Parch    Ticket      Fare        Cabin Embarked
7        3      1    349909   21.0750      Unknown        S
16       4      1    382652   29.1250      Unknown        Q
24       3      1    349909   21.0750      Unknown        S
27       3      2     19950  263.0000  C23 C25 C27        S
50       4      1   3101295   39.6875      Unknown        S
59       5      2  CA 2144   46.9000      Unknown        S
63       3      2    347088   27.9000      Unknown        S
68       4      2   3101281    7.9250      Unknown        S
71       5      2  CA 2144   46.9000      Unknown        S
85       3      0   3101278   15.8500      Unknown        S
88       3      2     19950  263.0000  C23 C25 C27        S
119      4      2    347082   31.2750      Unknown        S
159      8      2  CA. 2343   69.5500      Unknown        S
164      4      1   3101295   39.6875      Unknown        S
171      4      1    382652   29.1250      Unknown        Q
176      3      1      4133   25.4667      Unknown        S
180      8      2  CA. 2343   69.5500      Unknown        S
182      4      2    347077   31.3875      Unknown        S
201      8      2  CA. 2343   69.5500      Unknown        S
229      3      1      4133   25.4667      Unknown        S
233      4      2    347077   31.3875      Unknown        S
261      4      2    347077   31.3875      Unknown        S
266      4      1   3101295   39.6875      Unknown        S
278      4      1    382652   29.1250      Unknown        Q
324      8      2  CA. 2343   69.5500      Unknown        S
341      3      2     19950  263.0000  C23 C25 C27        S
374      3      1    349909   21.0750      Unknown        S
386      5      2  CA 2144   46.9000      Unknown        S
409      3      1      4133   25.4667      Unknown        S
480      5      2  CA 2144   46.9000      Unknown        S
485      3      1      4133   25.4667      Unknown        S
541      4      2    347082   31.2750      Unknown        S
```

Ex3b: Model Planning and Building - Regression

Learning Objective:
The objective of this experiment is to enable students to plan, build, and evaluate regression models using a real-world dataset.

Problem Statement:
A marketing agency wants to help a retail company optimize its advertising budget. The company invests in three advertising channels—TV, Radio, and Newspaper—and wants to understand how these advertising expenditures influence product sales. By developing predictive models, the company aims to estimate future sales based on advertising investments and identify the most suitable regression model for accurate sales prediction.

As a Data Scientist, your task is to build and compare multiple regression models using the Advertising dataset. Evaluate the performance of each model using appropriate regression metrics and communicate the findings through visualizations to support business decision-making.

Tasks to Perform:
1. Download the Advertising Dataset from Kaggle and load it into a Pandas DataFrame. 2. Explore the dataset by: Displaying the first and last five records, Checking the dataset shape and data types, Identifying missing values and duplicate records, Displaying summary statistics. 3. Select: TV, Radio, Newspaper as Input Features (X), Sales as Target Variable (y). **Perform Exploratory Data Analysis**:
4. Generate a Pair Plot for the selected features.
5. Plot a Correlation Heatmap.
6. Identify the feature most strongly correlated with Sales.

Model Planning and Model Building
7. Split the dataset into Training (80%) and Testing (20%) dataset.
8. Build a Linear Regression model and predict Sales for the test dataset.
9. Build a Polynomial Regression model (Degree = 2) and predict Sales for the test dataset. 10. Build a Support Vector Regression (SVR) model with the RBF kernel and predict Sales for the test dataset.

Model Evaluation
11. Evaluate all three models using: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 Score
12. Compare the performance of all models using a single grouped bar chart. 13. Plot Actual vs Predicted Sales for the best-performing model.

Model Interpretation
14. Identify the model with the lowest prediction error.
15. Suggest **three recommendations** to help the company optimize its advertising budget based on the model results.

+ Code + Text

```
#EX:3
#NAME:Nithin S [241801191]
#step 1
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
```

```
from google.colab import files
import pandas as pd

uploaded = files.upload()

df = pd.read_csv('Advertising.csv')

print(df.head())
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Advertising.csv to Advertising.csv
   Unnamed: 0     TV  Radio  Newspaper  Sales
0           1  230.1   37.8       69.2   22.1
1           2   44.5   39.3       45.1   10.4
2           3   17.2   45.9       69.3    9.3
3           4  151.5   41.3       58.5   18.5
4           5  180.8   10.8       58.4   12.9
```

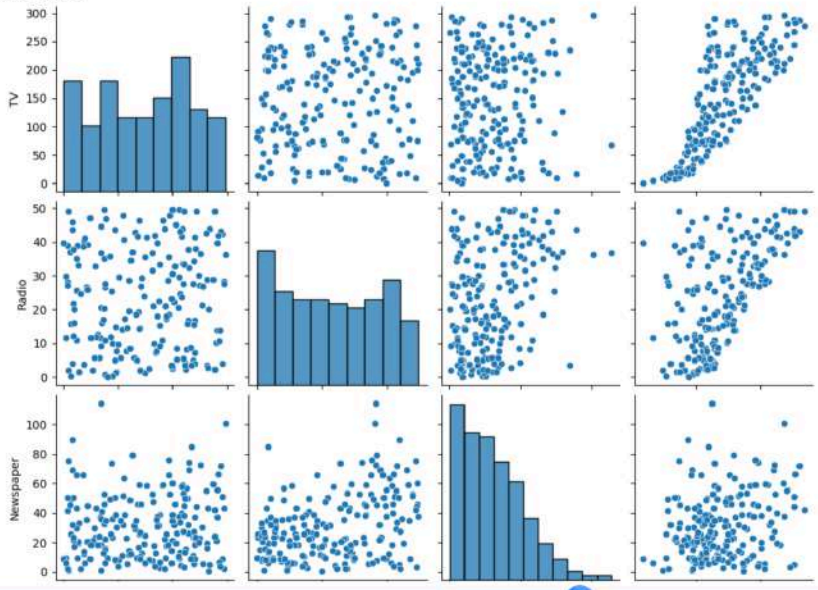
```
#EX:3
#NAME:Nithin S [241801191]
#step 2
print(df.info())
print(df.isnull().sum())
```

Terminal

```
#EX:3
#NAME:Nithin S [241801191]
#step 2
print(df.info())
print(df.isnull().sum())
print(df.dtypes)
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']
sns.pairplot(df[['TV', 'Radio', 'Newspaper', 'Sales']])
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Unnamed: 0  200 non-null    int64
 1   TV          200 non-null    float64
 2   Radio       200 non-null    float64
 3   Newspaper   200 non-null    float64
 4   Sales       200 non-null    float64
dtypes: float64(4), int64(1)
memory usage: 7.9 KB
None
Unnamed: 0    0
TV            0
Radio         0
Newspaper     0
Sales         0
dtype: int64
Unnamed: 0      int64
TV            float64
Radio         float64
Newspaper     float64
Sales         float64
dtype: object
```

```
TV          float64
Radio       float64
Newspaper   float64
Sales       float64
dtype: object
```



Terminal

```python
#EX:3
#NAME:Nithin S [241801191]
#step 3
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(X_train.shape)
print(X_test.shape)
```

```
(160, 3)
(40, 3)
```

```python
#EX:3
#NAME:Nithin S [241801191]
#step 4
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
print(y_pred_lr[:10])
```

```
[16.4080242  20.88988209 21.55384318 10.60850256 22.11237326 13.10559172
 21.05719192  7.46101034 13.60634581 15.15506967]
```

```python
#EX:3
#NAME:Nithin S [241801191]
#step 5
poly = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
poly.fit(X_train, y_train)
y_pred_poly = poly.predict(X_test)
print(y_pred_poly[:10])
```

```
[17.25443578 22.7193321  20.42799436  7.542709   24.37103037 12.55927164
 22.80784986  8.34371358 12.0586023  15.69024565]
```

```python
#EX:3
#NAME:Nithin S [241801191]
#step 6
```

Terminal

```
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
svr = make_pipeline(
    StandardScaler(),
    SVR(kernel='rbf')
)
svr.fit(X_train, y_train)
y_pred_svr = svr.predict(X_test)
print(y_pred_svr[:10])
```

```
[17.1879597  21.60497277 19.5802057   9.76637843 19.9549199  12.49691541
 21.90662014  8.54092114 12.05397226 15.48663882]
```

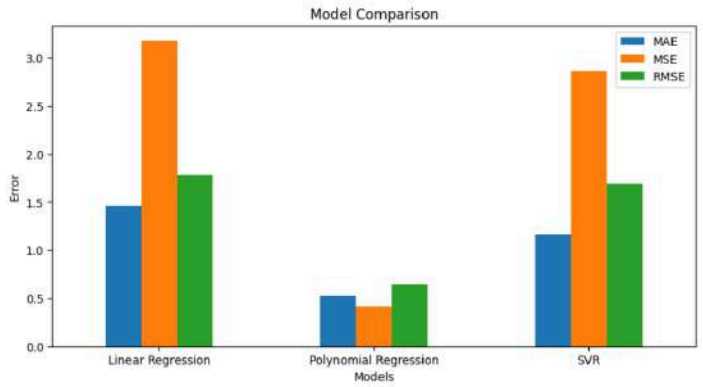
```
#EX:3
#NAME:Nithin S [241801191]
#step 7
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np
models = ['Linear Regression', 'Polynomial Regression', 'SVR']
predictions = [y_pred_lr, y_pred_poly, y_pred_svr]
mae = []
mse = []
rmse = []
for pred in predictions:
    mae.append(mean_absolute_error(y_test, pred))
    mse.append(mean_squared_error(y_test, pred))
    rmse.append(np.sqrt(mean_squared_error(y_test, pred)))

results = pd.DataFrame({
    'Model': models,
    'MAE': mae,
    'MSE': mse,
    'RMSE': rmse
})
print(results)
```

```
                 Model       MAE       MSE      RMSE
0    Linear Regression  1.460757  3.174097  1.781600
1  Polynomial Regression  0.526179  0.412910  0.642581
2                  SVR  1.162139  2.862109  1.691777
```

Terminal

```
#EX:3
#NAME:Nithin S [241801191]
#step 7.2
import matplotlib.pyplot as plt
results.set_index('Model')[['MAE', 'MSE', 'RMSE']].plot(
    kind='bar',
    figsize=(10,5)
)
plt.title('Model Comparison')
plt.xlabel('Models')
plt.ylabel('Error')
plt.xticks(rotation=0)
plt.show()
```



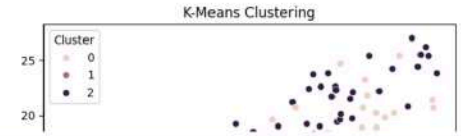
Terminal

```python
#EX:3
#NAME:Nithin S [241801191]
#step 8
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_scaled)
print(df.head())
```

```
   Unnamed: 0     TV  Radio  Newspaper  Sales  Cluster
0           1  230.1   37.8       69.2   22.1        2
1           2   44.5   39.3       45.1   10.4        2
2           3   17.2   45.9       69.3    9.3        2
3           4  151.5   41.3       58.5   18.5        2
4           5  180.8   10.8       58.4   12.9        2
```

```python
#EX:3
#NAME:Nithin S [241801191]
#step 9
sns.scatterplot(
    data=df,
    x='TV',
    y='Sales',
    hue='Cluster'
)

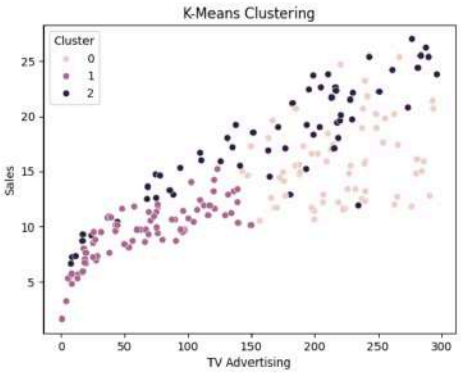
plt.title('K-Means Clustering')
plt.xlabel('TV Advertising')
plt.ylabel('Sales')
plt.show()
```



Terminal


```python
#EX:3
#NAME:Nithin S [241801191]
#step 9
sns.scatterplot(
    data=df,
    x='TV',
    y='Sales',
    hue='Cluster'
)

plt.title('K-Means Clustering')
plt.xlabel('TV Advertising')
plt.ylabel('Sales')
plt.show()
```

Terminal

Ex4: Information Retrieval using NLTK

Learning Objective:
The objective of this experiment is to enable students to understand the text preprocessing pipeline and implement a basic information retrieval system using TF-IDF vectorization, Cosine Similarity, NLTK.

Problem Statement:
An e-commerce company receives thousands of customer reviews for its products every day. Customer support executives often need to retrieve reviews that are similar to a user's search query (e.g., "great product with fast shipping" or "poor quality") to quickly understand customer feedback and address common concerns.

As an NLP Engineer, your task is to preprocess customer review text, convert the reviews into numerical feature vectors using the TF-IDF (Term Frequency–Inverse Document Frequency) technique, and develop a simple information retrieval system that returns the most relevant customer reviews for a given search query using Cosine Similarity.

Tasks to Perform:

1. Use Amazon Fine Food Reviews Dataset from Kaggle and load it into a Pandas DataFrame. 2. Install and import the required Python libraries:
3. Import the required NLTK resources:

Clean the dataset

4. Select the Text column containing customer reviews, Remove missing or null values, Remove duplicate reviews (if any), Retain only the first 10,000 reviews for faster processing. **Text Preprocessing**
5. Create a preprocessing function to perform the following operations on each review: Convert text to lowercase, Remove punctuation, numbers, and special characters, Tokenize the text into words, Remove English stopwords, Join the cleaned tokens into a single string.
6. Apply the preprocessing function to all reviews and store the cleaned reviews in a new column named Cleaned_Text.
7. Display the original and cleaned versions of any five reviews.
8. Convert the cleaned review text into TF-IDF feature vectors. Display: Shape of the TF-IDF matrix, Total number of unique terms (vocabulary size).

Information Retrieval

10. Implement a document retrieval function that: Accepts a user query, Applies the same preprocessing steps used for the reviews, Converts the query into a TF-IDF vector, Computes Cosine Similarity between the query and all review vectors, Retrieves the Top 5 most similar reviews, Display Similarity score, Original review, Cleaned review.

Query Evaluation

11. Test the retrieval system using the following sample queries: "great product with fast shipping", "disappointed", "excellent taste", "poor packaging"
12. Compare the retrieved reviews for different queries and comment on their relevance.

```
#Name:Nithin S[241801191]
#ex:4
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
```

```
#Name:Nithin S[241801191]
#ex:4
#step 1  Load Dataset
from google.colab import files
uploaded = files.upload()
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving House Price Prediction Dataset.csv to House Price Prediction Dataset.csv
```

```
df = pd.read_csv("House Price Prediction Dataset.csv")
print("First 5 rows:")
print(df.head())
```

```
First 5 rows:
   Id  Area  Bedrooms  Bathrooms  Floors  YearBuilt  Location  Condition  \
0   1  1360         5          4       3       1970  Downtown  Excellent
1   2  4272         5          4       3       1958  Downtown  Excellent
2   3  3592         2          2       3       1938  Downtown       Good
3   4   966         4          2       2       1902  Suburban       Fair
4   5  4926         1          4       2       1975  Downtown       Fair

  Garage   Price
0     No  149919
1     No  424998
2     No  266746
```

Terminal

```
#Name:Nithin S[241801191]
#ex:4
#step 2
print("\nDataset Information:")
print(df.info())
print("\nStatistical Description:")
print(df.describe())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Id         2000 non-null   int64
 1   Area       2000 non-null   int64
 2   Bedrooms   2000 non-null   int64
 3   Bathrooms  2000 non-null   int64
 4   Floors     2000 non-null   int64
 5   YearBuilt  2000 non-null   int64
 6   Location   2000 non-null   object
 7   Condition  2000 non-null   object
 8   Garage     2000 non-null   object
 9   Price      2000 non-null   int64
dtypes: int64(7), object(3)
memory usage: 156.4+ KB
None

Statistical Description:
                Id         Area     Bedrooms   Bathrooms       Floors  \
count  2000.000000  2000.000000  2000.000000  2000.00000  2000.000000
mean   1000.500000  2786.209500     3.003500     2.55250     1.993500
std     577.494589  1295.146799     1.424606     1.10899     0.809188
min       1.000000   501.000000     1.000000     1.00000     1.000000
25%     500.750000  1653.000000     2.000000     2.00000     1.000000
50%    1000.500000  2833.000000     3.000000     3.00000     2.000000
75%    1500.250000  3887.500000     4.000000     4.00000     3.000000
max    2000.000000  4999.000000     5.000000     4.00000     3.000000

         YearBuilt          Price
count  2000.000000    2000.000000
mean   1961.446000  537676.855000
```

Terminal

```
#Name:Nithin S[241801191]
#ex:4
#step 3
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Id            0
Area          0
Bedrooms      0
Bathrooms     0
Floors        0
YearBuilt     0
Location      0
Condition     0
Garage        0
Price         0
dtype: int64
```

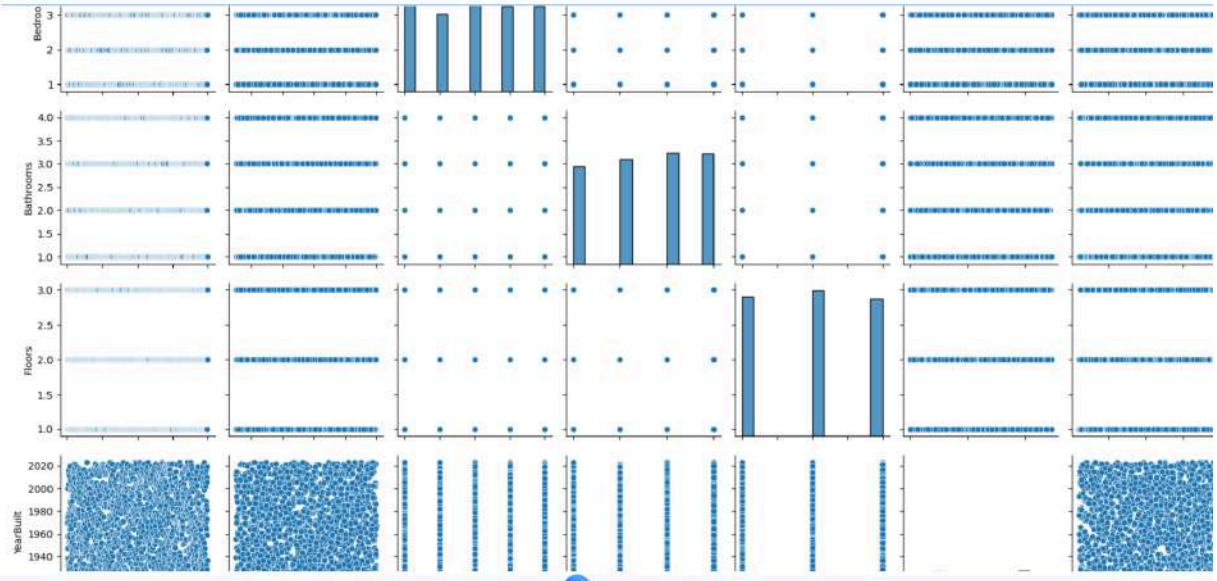
```
#Name:Nithin S[241801191]
#ex:4
#step 4
X = df[['Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt']]
y = df['Price']
print("\nFeatures:")
print(X.head())
print("\nTarget:")
print(y.head())
```

```
Features:
   Area  Bedrooms  Bathrooms  Floors  YearBuilt
0  1360         5          4       3       1970
1  4272         5          4       3       1958
2  3592         2          2       3       1938
3   966         4          2       2       1902
4  4926         1          4       2       1975

Target:
0    149919
1    424998
2    266746
```

Terminal

```
#Name:Nithin S[241801191]
#ex:4
#step 5
sns.pairplot(df)
plt.show()
```



Terminal

```python
#Name:Nithin S[241801191]
#ex:4
#step 6
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```python
#Name:Nithin S[241801191]
#ex:4
#step 7
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)
```

```python
#Name:Nithin S[241801191]
#ex:4
#step 8
polynomial_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
polynomial_model.fit(X_train, y_train)
y_pred_poly = polynomial_model.predict(X_test)
```

```python
#Name:Nithin S[241801191]
#ex:4
#step 9
svr_model = SVR(kernel='rbf')
svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)
```

```python
#Name:Nithin S[241801191]
#ex:4
#step 10
```

Terminal

```python
#step 10
models = {
    "Linear Regression": y_pred_linear,
    "Polynomial Regression": y_pred_poly,
    "SVR": y_pred_svr
}
for name, prediction in models.items():
    mae = mean_absolute_error(y_test, prediction)
    mse = mean_squared_error(y_test, prediction)
    rmse = np.sqrt(mse)
    print("\nModel:", name)
    print("MAE :", mae)
    print("MSE :", mse)
    print("RMSE:", rmse)
```

```
Model: Linear Regression
MAE : 244587.1999197856
MSE : 78789724000.88458
RMSE: 280695.0729900412

Model: Polynomial Regression
MAE : 245319.05662903108
MSE : 79234323974.83661
RMSE: 281485.9214504992

Model: SVR
MAE : 242647.33691047208
MSE : 77902605845.91304
RMSE: 279110.3829059626
```

```python
#Name:Nithin S[241801191]
#ex:4
#step 11
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
kmeans.fit(X)
df['Cluster'] = kmeans.labels_
print("\nCluster Labels:")
print(df[['Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt', 'Cluster']])
```

Terminal

```python
#Name:Nithin S[241801191]
#ex:4
#step 11
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
kmeans.fit(X)
df['Cluster'] = kmeans.labels_
print("\nCluster Labels:")
print(df[['Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt', 'Cluster']])
```

```
Cluster Labels:
      Area  Bedrooms  Bathrooms  Floors  YearBuilt  Cluster
0     1360         5          4       3       1970        0
1     4272         5          4       3       1958        1
2     3592         2          2       3       1938        1
3      966         4          2       2       1902        0
4     4926         1          4       2       1975        1
...    ...       ...        ...     ...        ...      ...
1995  4994         5          4       3       1923        1
1996  3046         5          2       1       2019        2
1997  1062         5          1       2       1903        0
1998  4062         3          1       2       1936        1
1999  2989         5          1       3       1903        2

[2000 rows x 6 columns]
```

```python
#Name:Nithin S[241801191]
#ex:4
#step 12
sns.scatterplot(
    data=df,
    x='Area',
    y='Price',
    hue='Cluster'
)
plt.xlabel("House Area")
plt.ylabel("House Price")
plt.title("House Price Clusters")
```

Terminal

```python
#Name:Nithin S[241801191]
#ex:4
#step 12
sns.scatterplot(
    data=df,
    x='Area',
    y='Price',
    hue='Cluster'
)
plt.xlabel("House Area")
plt.ylabel("House Price")
plt.title("House Price Clusters")
plt.show()
```



Terminal

Ex5: Exploratory Data Analysis

Learning Objective:
The objective of this experiment is to enable students to understand and perform Exploratory Data Analysis (EDA)on a real-world Netflix titles dataset using Python.

Problem Statement:
Netflix maintains a large collection of movies and TV shows containing information such as title, content type, release year, date added, country, rating, duration, genre, director, cast, and description. Netflix wants to understand the characteristics and growth of its content library and identify important patterns in its movies and TV shows.

As a Data Scientist, perform an Exploratory Data Analysis on the Netflix Titles dataset. Analyse the content distribution, ratings, release and addition trends, duration, countries, genres, directors, actors, and textual descriptions. Use suitable statistical techniques and visualisations to identify patterns and generate meaningful insights from the data.

Tasks to Perform:

1. Download Netflix_titles dataset from kaggle and convert it to dataframe. 2. Explore the Dataset
3. **Content Distribution Analysis. (Plot graphs wherever needed)**
    a. How many Movies vs TV Shows are in the dataset?
    b. What is the distribution of ratings (e.g., TV-MA, PG, R, etc.)?
    c. Which countries have the most Netflix content?
4. **Time-Based Trend Analysis (Plot graphs wherever needed)**
    a. In which years were the most shows/movies released?
    b. In which years were titles added to Netflix?
    c. How has Netflix's content growth trend changed over time?
5. **Duration & Content Type Analysis (Plot graphs wherever needed)** a. What is the distribution of movie durations (in minutes)?
    b. How many TV shows exist per number of seasons?
6. **Genre & Categories Analysis (Plot graphs wherever needed)**
    a. What are the top 10 most common genres on Netflix?
    b. Which directors have the most titles on Netflix?
    c. Which actors appear most frequently across Netflix shows/movies?
7. **Text & Descriptions**
    a. What are the most common words in the description field? (word cloud) 8. In which months of the year does Netflix add the most content?
9. Which countries produce more movies vs TV shows**?**
10. What are the most common ratings for Movies vs TV Shows?
11. What are the oldest and newest titles available on Netflix?
12. Which genres frequently co-occur together**?** (Genre correlation heatmap)

```
# Step 1 - Name:Nithin S [241801191] - Ex:5
import pandas as pd
from google.colab import files
uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])
df.head()
```

Choose Files | No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving netflix_titles_2021.csv to netflix_titles_2021 (2).csv

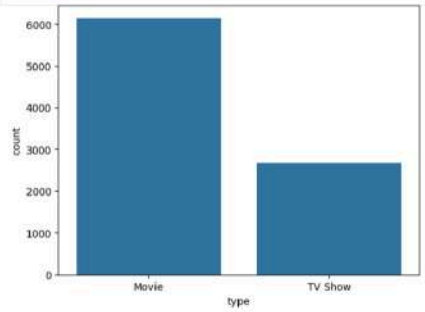
	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	descripti
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his lif filmm
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cap Town t
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...	To protect his family from a powerf drug lor
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet talk go dow amo
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a city of coaching centers known train

```
# Step 2 - Name:Nithin S [241801191] - Ex:5
print(df.shape)
print(df.info())
print(df.isnull().sum())
print(df.describe(include='all'))
```

```
(8807, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       8807 non-null   object
 1   type          8807 non-null   object
 2   title         8807 non-null   object
 3   director      6173 non-null   object
 4   cast          7982 non-null   object
 5   country       7976 non-null   object
 6   date_added    8797 non-null   object
 7   release_year  8807 non-null   int64
 8   rating        8803 non-null   object
 9   duration      8804 non-null   object
10   listed_in     8807 non-null   object
```

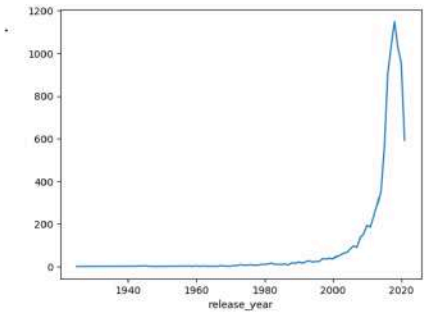
Terminal

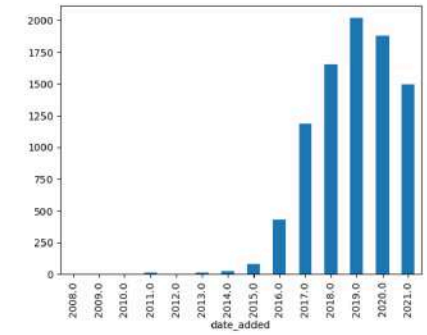
```
# Step 3 - Name:Nithin S [241801191] - Ex:5
import seaborn as sns
import matplotlib.pyplot as plt
print(df['type'].value_counts())
print(df['rating'].value_counts())
sns.countplot(data=df, x='type')
plt.show()
df['rating'].value_counts().head(10).plot(kind='bar')
plt.show()
df['country'].dropna().str.split(', ').explode().value_counts().head(10).plot(kind='bar')
plt.show()
```





Terminal

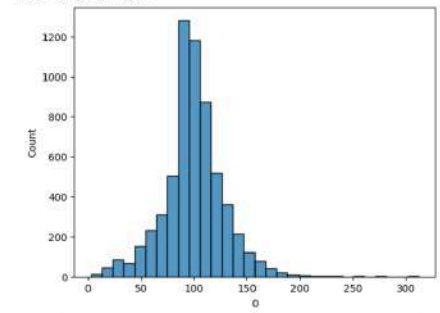




Terminal

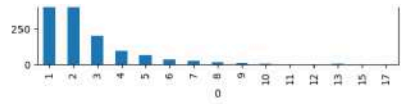
```
# Step 5 - Name:Nithin S [241801191] - Ex:5
movies = df[df['type'] == 'Movie']
duration = pd.to_numeric(movies['duration'].str.extract(r'(\d+)')[0], errors='coerce')
print(duration.describe())
sns.histplot(duration.dropna(), bins=30)
plt.show()
seasons = df[df['type'] == 'TV Show']['duration'].str.extract(r'(\d+)')[0].dropna().astype(int)
seasons.value_counts().sort_index().plot(kind='bar')
plt.show()
```

```
std        28.290593
min         3.000000
25%        87.000000
50%        98.000000
75%       114.000000
max       312.000000
Name: 0, dtype: float64
```



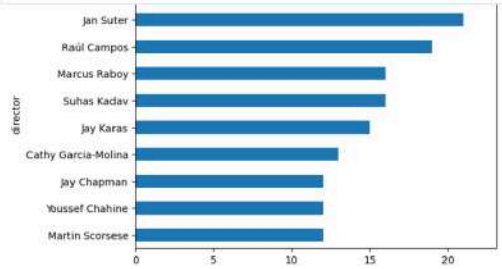


Terminal



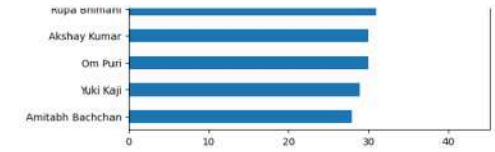

```
# Step 6 - Name:Nithin S [241801191] - Ex:5

for col in ['listed_in', 'director', 'cast']:
    top = df[col].dropna().str.split(', ').explode().value_counts().head(10)
    print(top)
    top.sort_values().plot(kind='barh')
    plt.show()
```




```
cast
Anupam Kher          43
Shah Rukh Khan       35
Julie Tejwani        33
Takahiro Sakurai     32
Naseeruddin Shah     32
Rupa Bhimani         31
Om Puri              30
Akshay Kumar         30
Yuki Kaji            29
Amitabh Bachchan     28
Name: count, dtype: int64
```

Terminal




```
# Step 7 - Name:Nithin S [241801191] - Ex:5

from wordcloud import WordCloud
text = ' '.join(df['description'].dropna())
WordCloud(width=800, height=400, background_color='white').generate(text).to_image()
plt.imshow(WordCloud(width=800, height=400, background_color='white').generate(text))
plt.axis('off')
plt.show()
```



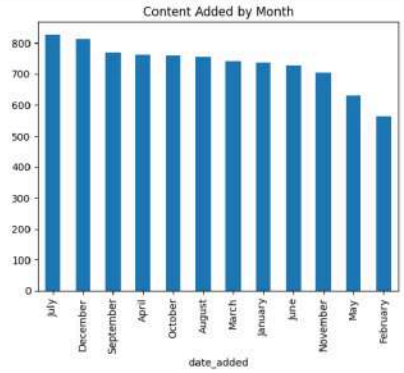

```
# Step 8 - Name:Nithin S [241801191]- Ex:5
df['date_added'].dt.month_name().value_counts().plot(kind='bar')
plt.title("Content Added by Month")
plt.show()
```



Terminal




```
# Step 8 - Name:Nithin S [241801191]- Ex:5
df['date_added'].dt.month_name().value_counts().plot(kind='bar')
plt.title("Content Added by Month")
plt.show()
```

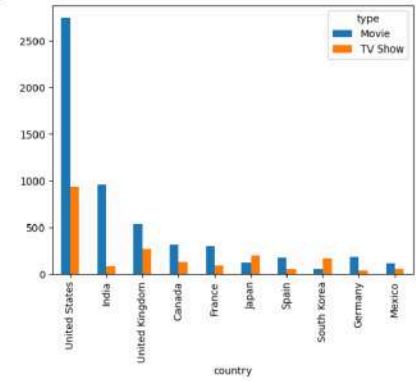



```
# Step 9 - Name:Nithin S [241801191] - Ex:5
c = df.dropna(subset=['country']).assign(country=lambda x: x['country'].str.split(', ')).explode('country')
ct = pd.crosstab(c['country'], c['type'])
ct['Total'] = ct.sum(axis=1)
ct.nlargest(10, 'Total')[['Movie', 'TV Show']].plot(kind='bar')
plt.show()
```



Terminal

```
# Step 9 - Name:Nithin S [241801191] - Ex:5
c = df.dropna(subset=['country']).assign(country=lambda x: x['country'].str.split(', ')).explode('country')
ct = pd.crosstab(c['country'], c['type'])
ct['Total'] = ct.sum(axis=1)
ct.nlargest(10, 'Total')[['Movie', 'TV Show']].plot(kind='bar')
plt.show()
```



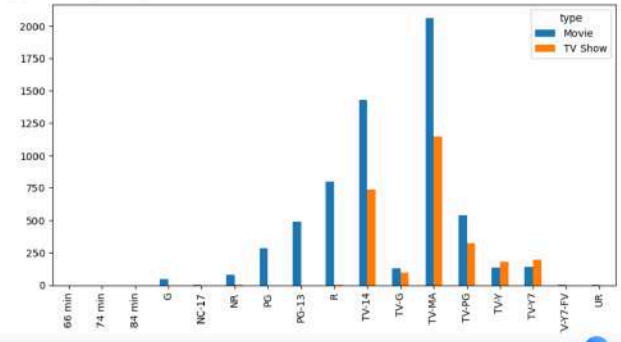

```
# Step 10 - Name:Nithin S [241801191]- Ex:5
r = pd.crosstab(df['rating'], df['type'])
print(r)
r.plot(kind='bar', figsize=(10,5))
plt.show()
```

```
type      Movie  TV Show
rating
66 min        1        0
74 min        1        0
```

Terminal

```
# Step 10 - Name:Nithin S [241801191]- Ex:5
r = pd.crosstab(df['rating'], df['type'])
print(r)
r.plot(kind='bar', figsize=(10,5))
plt.show()
```

```
type      Movie  TV Show
rating
66 min        1        0
74 min        1        0
84 min        1        0
G            41        0
NC-17         3        0
NR           75        5
PG          287        0
PG-13       490        0
R           797        2
TV-14      1427      733
TV-G        126       94
TV-MA      2062     1145
TV-PG       540      323
TV-Y        131      176
TV-Y7       139      195
TV-Y7-FV      5        1
UR            3        0
```



Terminal

```
# Step 11 - Name:Nithin S [241801191] - Ex:5

print(df.loc[df['release_year'].idxmin(), ['title', 'release_year']])
print(df.loc[df['release_year'].idxmax(), ['title', 'release_year']])
```

```
title           Pioneers: First Women Filmmakers*
release_year                                 1925
Name: 4250, dtype: object
title           Blood & Water
release_year             2021
Name: 1, dtype: object
```

```
# Step 12 - Name:Nithin S [241801191] - Ex:5

from sklearn.preprocessing import MultiLabelBinarizer
g = df['listed_in'].dropna().str.split(', ')
mlb = MultiLabelBinarizer()
m = pd.DataFrame(mlb.fit_transform(g), columns=mlb.classes_)
top = m.sum().nlargest(10).index
corr = m[top].T.dot(m[top])
plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, fmt='d', cmap='Blues')
plt.show()
```



Terminal

Ex6: Time Series Data Analysis and Forecast

Learning Objective:
The objective of this experiment is to enable students to understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement:
An airline company maintains monthly records of the number of passengers travelling by air. The company wants to understand historical passenger trends and seasonal patterns and use this information to forecast future passenger demand.

As a Data Scientist, analyse the historical monthly airline passenger data, determine whether the time series is stationary, apply appropriate transformations when necessary, build an ARIMA forecasting model, predict passenger numbers for the test period, and evaluate the accuracy of the forecasts.

Tasks to Perform:

1. Download airline-passenges dataset from https://github.com/jbrownlee/Datasets/blob/master/airline-passengers.csv 2. Read the CSV with parse_dates and index_col so the time index becomes a DatetimeIndex.
3. Explore the Dataset.
4. Plot the series to inspect trend/seasonality/outliers.
5. Do Stationarity check by running Augmented Dickey-Fuller (ADF) test. 6. If the series is non-stationary, convert it to stationary by differencing the series. 7. Re-run ADF on the differenced series and visualize the new dataframe. 8. If still non-stationary, consider another difference (d=2) or transformation (log) before differencing.
9. Split the dataset for training and testing on original series.
10. Fit ARIMA model on training set. d value comes from Step 8 (how many differences you applied).
11. Perform forecasts on the test dataset.
12. Compute metrics MSE, MAE, RMSE on the predictions.
13. Plot train, test, and predictions in a single plot.

```python
# Step 1 - Name:Nithin S[241801191] - Ex:5B

from google.colab import files
import pandas as pd
uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])
df.head()
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving airline-passengers.csv to airline-passengers.csv
```

	Month	Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121

```python
# Step 2 - Name:Nithin S[241801191] - Ex:5B

df = pd.read_csv(
    list(uploaded.keys())[0],
    parse_dates=['Month'],
    index_col='Month'
)
df.head()
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

```python
# Step 3 - Name:Nithin [241801191] - Ex:5B
```

```python
# Step 3 - Name:Nithin [241801191] - Ex:5B

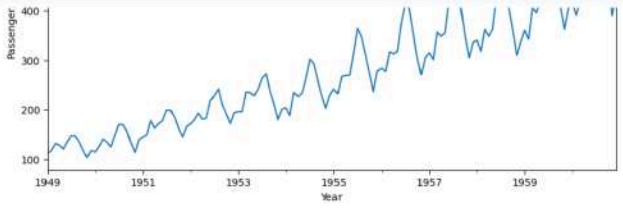
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

```
(144, 1)
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
None
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000
Passengers    0
dtype: int64
```

```python
# Step 4 - Name:Nithin S [241801191] - Ex:5B

import matplotlib.pyplot as plt
df.plot(figsize=(10,5))
plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Passengers")
plt.show()
```






```
# Step 5 - Name:Nithin S [241801191] - Ex:5B

from statsmodels.tsa.stattools import adfuller
result = adfuller(df['Passengers'])
print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.9918802434376411
/tmp/ipykernel_1984/2905564206.py:4: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, t
  result = adfuller(df['Passengers'])
```

```
# Step 6 - Name:Nithin S [241801191] - Ex:5B

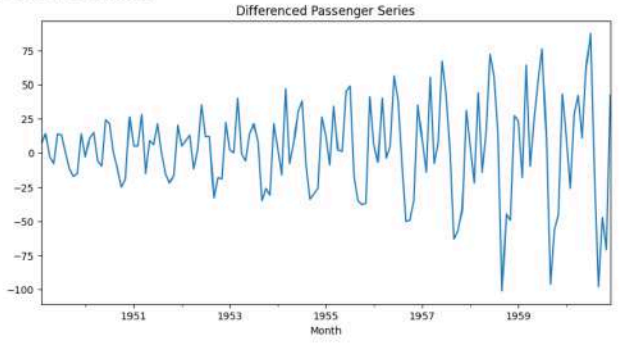
diff = df['Passengers'].diff().dropna()
print(diff.head())
```

```
Month
1949-02-01    6.0
1949-03-01   14.0
1949-04-01   -3.0
1949-05-01   -8.0
1949-06-01   14.0
Name: Passengers, dtype: float64
```

```
# Step 7 - Name:Nithin S [241801191] - Ex:5B
result = adfuller(diff)
print("ADF Statistic:", result[0])
print("p-value:", result[1])
diff.plot(figsize=(10,5))
plt.title("Differenced Passenger Series")
```

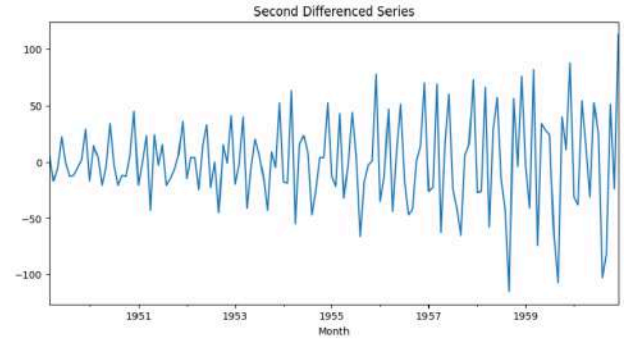
```
# Step 7 - Name:Nithin S [241801191] - Ex:5B
result = adfuller(diff)
print("ADF Statistic:", result[0])
print("p-value:", result[1])
diff.plot(figsize=(10,5))
plt.title("Differenced Passenger Series")
plt.show()
```

```
/tmp/ipykernel_1984/4064224130.py:3: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, t
  result = adfuller(diff)
ADF Statistic: -2.829266824170007
p-value: 0.05421329028382478
```




```
# Step 8 - Name:Nithin S [241801191] - Ex:5B
diff2 = diff.diff().dropna()
result = adfuller(diff2)
print("ADF Statistic:", result[0])
print("p-value:", result[1])
diff2.plot(figsize=(10,5))
plt.title("Second Differenced Series")
plt.show()
```

```
/tmp/ipykernel_1984/2335684628.py:4: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whichever is later, t
  result = adfuller(diff2)
ADF Statistic: -16.384231542468513
p-value: 2.7328918500142026e-29
```




```
# Step 9 - Name:Nithin S [241801191]  - Ex:5B

train = df['Passengers'][:-12]
test = df['Passengers'][-12:]
print("Training:", len(train))
print("Testing:", len(test))
```

```
Training: 132
Testing: 12
```

```
# Step 10 - Name:Nithin S [241801191]  - Ex:5B
from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(train, order=(1,1,1))
model_fit = model.fit()
print(model_fit.summary())
```

```
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
                               SARIMAX Results
==============================================================================
Dep. Variable:              Passengers   No. Observations:                  132
Model:                 ARIMA(1, 1, 1)   Log Likelihood               -625.518
Date:                Tue, 06 Oct 2026   AIC                           1257.035
Time:                        03:24:34   BIC                           1265.661
Sample:                    01-01-1949   HQIC                          1260.540
                         - 12-01-1959
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.5436      0.100     -5.443      0.000      -0.739      -0.348
ma.L1          0.9272      0.051     18.336      0.000       0.828       1.026
sigma2       816.5398     95.147      8.582      0.000     630.056    1003.024
===================================================================================
Ljung-Box (L1) (Q):                   0.31   Jarque-Bera (JB):                 2.51
Prob(Q):                              0.57   Prob(JB):                         0.29
Heteroskedasticity (H):               6.76   Skew:                            -0.27
Prob(H) (two-sided):                  0.00   Kurtosis:                         3.42
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

```
# Step 11 - Name:Nithin S [241B01191] - Ex:5B

forecast = model_fit.forecast(steps=len(test))
print(forecast)
```

```
1960-01-01    433.451927
1960-02-01    417.984221
1960-03-01    426.393139
1960-04-01    421.821685
1960-05-01    424.306927
1960-06-01    422.955841
1960-07-01    423.690350
1960-08-01    423.291039
1960-09-01    423.508122
1960-10-01    423.390106
1960-11-01    423.454265
1960-12-01    423.419385
Freq: MS, Name: predicted_mean, dtype: float64
```

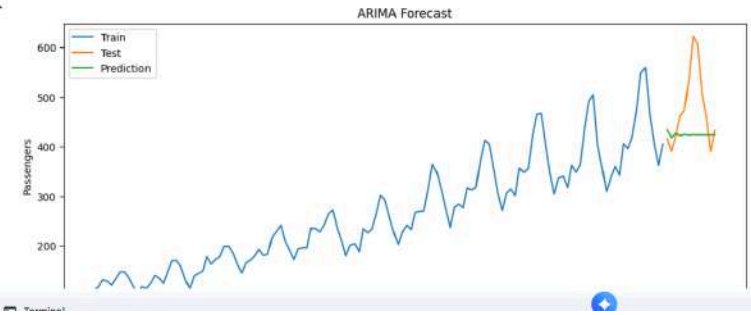
```
# Step 12 - Name:Nithin S [241B01191]  - Ex:5B

from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np
mse = mean_squared_error(test, forecast)
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)
print("MSE:", mse)
print("MAE:", mae)
print("RMSE:", rmse)
```

```
MSE: 8322.701142131522
MAE: 66.24167467277367
RMSE: 91.2288394211585
```

```
# Step 13 - Name:Nithin S [241B01191]- Ex:5B

plt.figure(figsize=(12,5))
plt.plot(train, label='Train')
plt.plot(test, label='Test')
plt.plot(test.index, forecast, label='Prediction')
plt.title("ARIMA Forecast")
plt.xlabel("Year")
plt.ylabel("Passengers")
plt.legend()
plt.show()
```



Ex3a: Model Planning and Building - Classification

Learning Objective:
The objective of this experiment is to enable students to plan, build, and evaluate basic machine learning models using real-world datasets.

Problem Statement:
A telecommunications company has noticed that a growing number of customers are discontinuing its services, resulting in reduced revenue and increased customer acquisition costs. After completing data preprocessing and exploratory analysis, the company now aims to develop predictive models that can identify customers who are likely to churn. Early prediction of customer churn will enable the company to implement targeted retention strategies, improve customer satisfaction, and reduce business losses.

As a Data Scientist, your task is to build and compare multiple classification models to predict whether a customer will churn. You will train different machine learning algorithms, evaluate their performance using appropriate classification metrics, and identify the most effective model for customer churn prediction. Finally, interpret the important factors influencing customer churn and communicate your findings through suitable visualizations and performance comparisons.

Tasks to Perform:

1. Use CustomerChurn_Preprocessed.csv file which you created in Ex2. 2. Split the cleaned dataset into Training (80%) and Testing (20%) datasets. **Model Building**
3. Train a Logistic Regression classifier and predict the customer churn on the test dataset. 4. Train a Decision Tree Classifier and predict the customer churn on the test dataset. 5. Train a Random Forest Classifier and predict the customer churn on the test dataset. **Model Evaluation**
6. Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, F1-Score. 7. Compare the performance of all three models using a single bar chart showing the evaluation metrics.
8. Identify the best-performing model based on the evaluation results.

Business Recommendation

9. Based on the best-performing model, provide three recommendations that the telecommunications company can implement to reduce customer churn.

```
# Name: Nithin S
# Roll No: 241801191

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score, classification_report
)
```

```
# Name: Nithin S
# Roll No: 241801191

from google.colab import files

uploaded = files.upload()
df = pd.read_csv('CustomerChurn_Preprocessed.csv')

print("Dataset Shape:", df.shape)
display(df.head())
df.info()
```

```
Choose Files  CustomerC...ocessed.csv
CustomerChurn_Preprocessed.csv(text/csv) - 3282220 bytes, last modified: 10/13/2026 - 100% done
Saving CustomerChurn_Preprocessed.csv to CustomerChurn_Preprocessed.csv
Dataset Shape: (64374, 12)
```

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Contract Length	Total Spend	Last Interaction	Churn
0	1	22	Female	25	14	4	27	Basic	Monthly	598	9	1
1	2	41	Female	28	28	7	13	Standard	Monthly	584	20	0
2	3	47	Male	27	10	2	29	Premium	Annual	757	21	0
3	4	35	Male	9	12	5	17	Premium	Quarterly	232	18	0
4	5	53	Female	58	24	9	2	Standard	Annual	533	18	0

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 64374 entries, 0 to 64373
Data columns (total 12 columns):
```

```
# Name: Nithin S
# Roll No: 241801191

df = df.drop(columns=['customerID'], errors='ignore')

if df['Churn'].dtype == 'object':
    df['Churn'] = df['Churn'].astype(str).str.strip().map(
        {'Yes': 1, 'No': 0}
    )

df = df.dropna(subset=['Churn'])

X = pd.get_dummies(df.drop(columns=['Churn']), drop_first=True)
y = df['Churn'].astype(int)

X = X.replace([np.inf, -np.inf], np.nan)
X = X.fillna(X.median())

print("Features:", X.shape)
print("Target Distribution:\n", y.value_counts())
```

```
Features: (64374, 13)
Target Distribution:
 Churn
0    33881
1    30493
Name: count, dtype: int64
```

```
# Name: Nithin S
# Roll No: 241801191

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (51499, 13)
Testing Data: (12875, 13)
```

```
# Name: Nithin S
# Roll No: 241801191

lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000, random_state=42)
)
```

```python
# Name: Nithin S
# Roll No: 241801191

lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000, random_state=42)
)

lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)

print("Logistic Regression")
print(classification_report(y_test, lr_pred, zero_division=0))
```

```
Logistic Regression
              precision    recall  f1-score   support

           0       0.87      0.87      0.87      6776
           1       0.85      0.85      0.85      6099

    accuracy                           0.86     12875
   macro avg       0.86      0.86      0.86     12875
weighted avg       0.86      0.86      0.86     12875
```

```python
# Name: Nithin S
# Roll No: 241801191

dt = DecisionTreeClassifier(max_depth=10, random_state=42)

dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

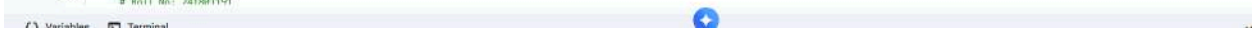
print("Decision Tree")
print(classification_report(y_test, dt_pred, zero_division=0))
```

```
Decision Tree
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      6776
           1       1.00      1.00      1.00      6099

    accuracy                           1.00     12875
   macro avg       1.00      1.00      1.00     12875
weighted avg       1.00      1.00      1.00     12875
```

```python
# Name: Nithin S
# Roll No: 241801191
```



```python
# Name: Nithin S
# Roll No: 241801191

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight='balanced'
)

rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

print("Random Forest")
print(classification_report(y_test, rf_pred, zero_division=0))
```

```
Random Forest
              precision    recall  f1-score   support

           0       0.99      1.00      1.00      6776
           1       1.00      0.99      1.00      6099

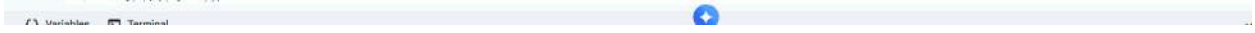
    accuracy                           1.00     12875
   macro avg       1.00      1.00      1.00     12875
weighted avg       1.00      1.00      1.00     12875
```

```python
# Name: Nithin S
# Roll No: 241801191

models = {
    'Logistic Regression': lr_pred,
    'Decision Tree': dt_pred,
    'Random Forest': rf_pred
}

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, (name, pred) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, pred, labels=[0, 1])
    sns.heatmap(
        cm, annot=True, fmt='d', cmap='Blues',
        xticklabels=['No Churn', 'Churn'],
        yticklabels=['No Churn', 'Churn'], ax=ax
    )
    ax.set_title(name)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
```



```
plt.show()
```




```
# Name: Nithin S
# Roll No: 241801191

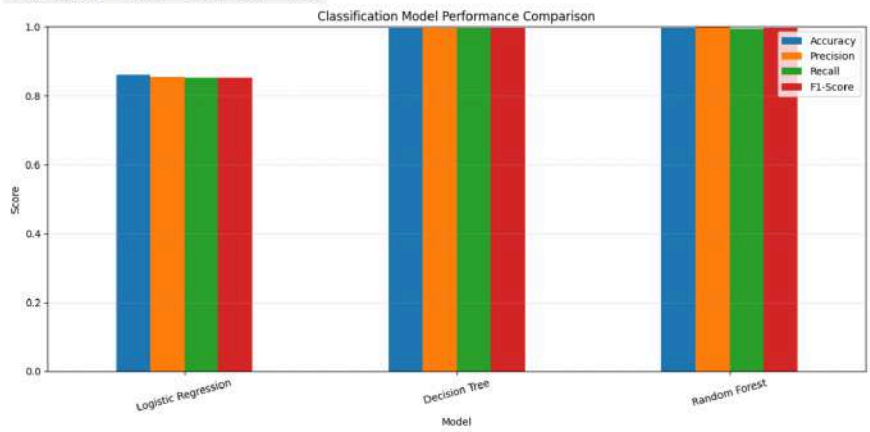
results = []

for name, pred in models.items():
    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, pred),
        'Precision': precision_score(y_test, pred, zero_division=0),
        'Recall': recall_score(y_test, pred, zero_division=0),
        'F1-Score': f1_score(y_test, pred, zero_division=0)
    })

results_df = pd.DataFrame(results).set_index('Model')
display(results_df.round(4))

results_df.plot(kind='bar', figsize=(12, 6), ylim=(0, 1))
plt.title('Classification Model Performance Comparison')
plt.ylabel('Score')
plt.xticks(rotation=15)
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.8609	0.8544	0.8515	0.8529
Decision Tree	0.9964	0.9961	0.9964	0.9962
Random Forest	0.9966	0.9995	0.9933	0.9964




```
Best Model: Random Forest
```

```
# Name: Nithin S
# Roll No: 241801191

best_model = {
    'Logistic Regression': lr,
```

```python
# Name: Nithin S
# Roll No: 241801191

best_model = {
    'Logistic Regression': lr,
    'Decision Tree': dt,
    'Random Forest': rf
}[best_model_name]

if best_model_name == 'Logistic Regression':
    importance = np.abs(
        best_model.named_steps['logisticregression'].coef_[0]
    )
else:
    importance = best_model.feature_importances_

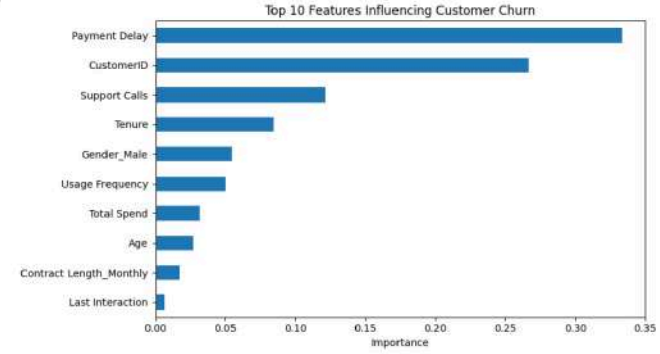
feature_importance = pd.Series(
    importance, index=X.columns
).sort_values(ascending=False)

print("Top 10 Important Features:")
print(feature_importance.head(10))

feature_importance.head(10).sort_values().plot(
    kind='barh', figsize=(9, 5)
)
plt.title('Top 10 Features Influencing Customer Churn')
plt.xlabel('Importance')
plt.tight_layout()
plt.show()

print("\nBusiness Recommendations:")
print("1. Offer targeted discounts and loyalty benefits.")
print("2. Improve customer support and resolve service complaints.")
print("3. Provide suitable plans to customers at high risk of churn.")
```

```
Top 10 Important Features:
Payment Delay              0.333511
CustomerID                 0.266802
Support Calls              0.121568
Tenure                     0.084546
Gender_Male                0.054316
Usage Frequency            0.050593
Total Spend                0.031569
Age                        0.026718
Contract Length_Monthly    0.017284
Last Interaction           0.006582
dtype: float64
```




```
Business Recommendations:
1. Offer targeted discounts and loyalty benefits.
2. Improve customer support and resolve service complaints.
3. Provide suitable plans to customers at high risk of churn.
```

Ex7: Model Planning and Building - Clustering

Learning Objective:
The objective of this experiment is to enable students to understand and implement the K Means clustering algorithm for customer segmentation.

Problem Statement:
A retail company wants to group its customers based on their Annual Income and Spending Score to design personalized marketing campaigns. By identifying customers with similar purchasing behavior, the company can improve customer satisfaction, optimize promotional offers, and increase overall sales.

Your task is to analyze the customer dataset, apply the K-Means Clustering algorithm to identify customer segments, visualize the clusters, and recommend suitable marketing strategies for each segment.

Tasks to Perform:

1. Use Mall Customer Segmentation Dataset from Kaggle and load it into a Pandas DataFrame.
2. Explore the dataset by: Displaying the first and last five records, Checking the dataset shape and data types, Identifying missing values and duplicate records, Displaying summary statistics.
3. Select the important features for clustering: Annual Income (k$), Spending Score (1–100).
4. Standardize the selected features using StandardScaler for clustering. 5. Visualize the selected features using: Scatter Plot, Pair Plot.

Model Planning and Building

6. Determine the optimal number of clusters using the Elbow Method. Plot the Elbow Curve and identify the optimal value of K.
7. Train the K-Means Clustering model using the optimal number of clusters. Assign cluster labels to each customer, add as a new column in the dataset and save as a file. **Cluster Visualization**
8. Visualize the customer clusters using a scatter plot: X-axis: Annual Income, Y-axis: Spending Score, Different colors for each cluster, Highlight cluster centroids, Display the coordinates of the cluster centroids.

Cluster Analysis

9. Calculate the number of customers in each cluster.
10. Compute the average Annual Income and Spending Score for each cluster.

```
# Name: Nithin S
# Roll No: 241801191

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from google.colab import files
```

```
# Name: Nithin S
# Roll No: 241801191

uploaded = files.upload()
df = pd.read_csv('Mall_Customers (2).csv')

print("First Five Records:")
display(df.head())

print("Last Five Records:")
display(df.tail())
```

Choose Files | Mall_Customers (2).csv

Mall_Customers (2).csv(text/csv) - 3612 bytes, last modified: 10/14/2026 - 100% done

Saving Mall_Customers (2).csv to Mall_Customers (2).csv

First Five Records:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Female	28	15	35
1	2	Female	62	17	82
2	3	Male	60	18	70
3	4	Male	70	19	1
4	5	Male	52	19	10

Last Five Records:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
195	196	Female	63	134	2
196	197	Female	57	135	69
197	198	Female	57	135	83
198	199	Female	38	137	20

```
# Name: Nithin S
# Roll No: 241801191

print("Dataset Shape:", df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Records:", df.duplicated().sum())

print("\nSummary Statistics:")
display(df.describe(include='all'))
```

```
Dataset Shape: (200, 5)

Data Types:
CustomerID                int64
Gender                   object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object

Missing Values:
CustomerID                0
Gender                    0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64

Duplicate Records: 0

Summary Statistics:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200	200.000000	200.000000	200.000000
unique	NaN	2	NaN	NaN	NaN
top	NaN	Female	NaN	NaN	NaN
freq	NaN	102	NaN	NaN	NaN
mean	100.500000	NaN	45.635000	77.110000	51.500000
std	57.879185	NaN	15.383597	33.830763	28.970788

```
# Roll No: 241801191

X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

print("Selected Features:")
display(X.head())

print("\nFeature Shape:", X.shape)
```

```
Selected Features:
```

	Annual Income (k$)	Spending Score (1-100)
0	15	35
1	17	82
2	18	70
3	19	1
4	19	10

```
Feature Shape: (200, 2)
```

```
# Name: Nithin S
# Roll No: 241801191

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=['Annual Income (Standardized)',
             'Spending Score (Standardized)']
)

print("Standardized Features:")
display(X_scaled_df.head())
```

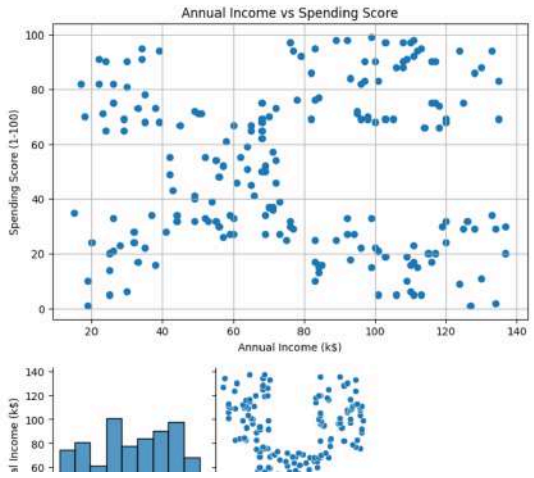
```
Standardized Features:
```

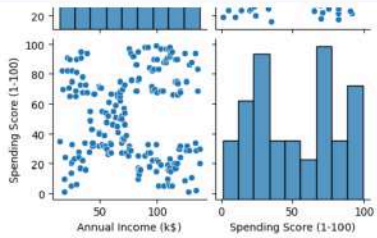
	Annual Income (Standardized)	Spending Score (Standardized)
0	-1.840510	-0.570968
1	-1.781244	1.055427
2	-1.751611	0.640177
3	-1.721978	-1.747509

```
# Name: Nithin S
# Roll No: 241801191

plt.figure(figsize=(8, 5))
plt.scatter(X['Annual Income (k$)'],
            X['Spending Score (1-100)'])
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Annual Income vs Spending Score')
plt.grid(True)
plt.show()

sns.pairplot(X)
plt.show()
```



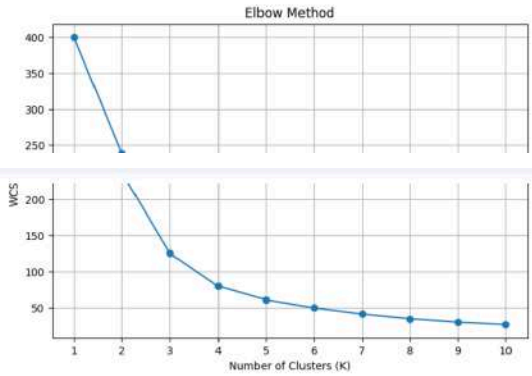



```
# Name: Nithin S
# Roll No: 241801191

wcss = []

for k in range(1, 11):
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X_scaled)
    wcss.append(model.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.title('Elbow Method')
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()
```




```
# Name: Nithin S
# Roll No: 241801191

optimal_k = 5

kmeans = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(X_scaled)

print("Cluster Labels Added:")
display(df.head())

df.to_csv('Mall_Customers_Clustered.csv', index=False)
files.download('Mall_Customers_Clustered.csv')
```

```
Cluster Labels Added:
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Female	28	15	35	4
1	2	Female	62	17	82	2
2	3	Male	60	18	70	2
3	4	Male	70	19	1	4
4	5	Male	52	19	10	4

```
# Name: Nithin S
```

4	5	Male	52	19	10	4

```
# Name: Nithin S
# Roll No: 241801191

centers_scaled = kmeans.cluster_centers_
centers = scaler.inverse_transform(centers_scaled)

plt.figure(figsize=(10, 6))

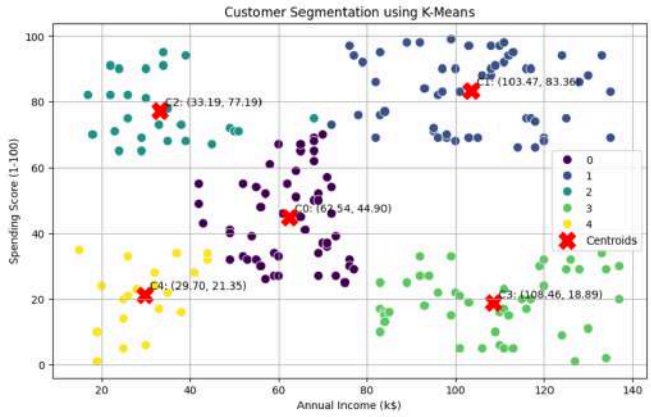
sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='viridis',
    s=80
)

plt.scatter(
    centers[:, 0],
    centers[:, 1],
    c='red',
    marker='X',
    s=250,
    label='Centroids'
)

for i, (income, score) in enumerate(centers):
    plt.annotate(
        f'C{i}: ({income:.2f}, {score:.2f})',
        (income, score),
        xytext=(5, 5),
        textcoords='offset points'
    )

plt.title('Customer Segmentation using K-Means')
plt.legend()
plt.grid(True)
plt.show()

print("Centroid Coordinates:")
for i, (income, score) in enumerate(centers):
    print(f"Cluster {i}: Income = {income:.2f}, "
          f"Spending Score = {score:.2f}")
```




```
Centroid Coordinates:
Cluster 0: Income = 62.54, Spending Score = 44.90
Cluster 1: Income = 103.47, Spending Score = 83.36
Cluster 2: Income = 33.19, Spending Score = 77.19
Cluster 3: Income = 108.46, Spending Score = 18.89
Cluster 4: Income = 29.70, Spending Score = 21.35
```

```
# Name: Nithin S
# Roll No: 241801191

print("Number of Customers in Each Cluster:")
print(df['Cluster'].value_counts().sort_index())

print("\nAverage Income and Spending Score:")
cluster_summary = df.groupby('Cluster')[
    ['Annual Income (k$)', 'Spending Score (1-100)']
].mean().round(2)
```

```
Cluster 4: Income = 29.70, Spending Score = 21.35
```

[30]

```
# Name: Nithin S
# Roll No: 241801191

print("Number of Customers in Each Cluster:")
print(df['Cluster'].value_counts().sort_index())

print("\nAverage Income and Spending Score:")
cluster_summary = df.groupby('Cluster')[
    ['Annual Income (k$)', 'Spending Score (1-100)']
].mean().round(2)

display(cluster_summary)

print("\nMarketing Recommendations:")

for cluster, row in cluster_summary.iterrows():
    income = row['Annual Income (k$)']
    score = row['Spending Score (1-100)']

    if income >= 60 and score >= 60:
        strategy = "Premium offers, loyalty rewards, and exclusive products."
    elif income >= 60 and score < 40:
        strategy = "Personalized promotions to encourage more spending."
    elif income < 40 and score >= 60:
        strategy = "Affordable deals, discounts, and value bundles."
    elif income < 40 and score < 40:
        strategy = "Budget-friendly offers and basic product promotions."
    else:
        strategy = "Targeted discounts and personalized recommendations."

    print(f"Cluster {cluster}: {strategy}")
```

```
Number of Customers in Each Cluster:
Cluster
0    52
1    55
2    27
3    46
4    20
Name: count, dtype: int64

Average Income and Spending Score:
```

Cluster	Annual Income (k$)	Spending Score (1-100)

[30]

```
    if income >= 60 and score >= 60:
        strategy = "Premium offers, loyalty rewards, and exclusive products."
    elif income >= 60 and score < 40:
        strategy = "Personalized promotions to encourage more spending."
    elif income < 40 and score >= 60:
        strategy = "Affordable deals, discounts, and value bundles."
    elif income < 40 and score < 40:
        strategy = "Budget-friendly offers and basic product promotions."
    else:
        strategy = "Targeted discounts and personalized recommendations."

    print(f"Cluster {cluster}: {strategy}")
```

```
Number of Customers in Each Cluster:
Cluster
0    52
1    55
2    27
3    46
4    20
Name: count, dtype: int64

Average Income and Spending Score:
```

Cluster	Annual Income (k$)	Spending Score (1-100)
0	62.54	44.90
1	103.47	83.36
2	33.19	77.19
3	108.46	18.89
4	29.70	21.35

```
Marketing Recommendations:
Cluster 0: Targeted discounts and personalized recommendations.
Cluster 1: Premium offers, loyalty rewards, and exclusive products.
Cluster 2: Affordable deals, discounts, and value bundles.
Cluster 3: Personalized promotions to encourage more spending.
Cluster 4: Budget-friendly offers and basic product promotions.
```

Next steps: Generate code with cluster_summary | New interactive sheet

Ex5: Information Extraction using spaCy

Learning Objective:
The objective of this experiment is to enable students to understand and implement information extraction techniques on unstructured customer review text using spaCy.

Problem Statement:
An e-commerce company receives thousands of customer reviews containing valuable information about products, brands, organizations, locations, quantities, and other entities. Manually identifying and organizing this information from large volumes of reviews is time consuming and inefficient.

As an NLP Engineer, your task is to process customer reviews using the spaCy Natural Language Processing library and develop an information extraction system that identifies and extracts meaningful linguistic information and named entities from the reviews. The extracted information should be organized into a structured format that can be further used for customer feedback analysis.

Tasks to Perform:

1. Reuse the Amazon Fine Food Reviews Dataset used in the previous experiment and load it into a Pandas DataFrame. Install and import the required libraries.

Clean the dataset

2. Select the Text column containing customer reviews, Remove missing or null values, Remove duplicate reviews (if any), Retain only the first 10,000 reviews for faster processing. **Text Preprocessing and Part-of-speech (POS) tags**

3. Apply the spaCy pipeline to the review text. For any five reviews, display: Original review, Tokens, Lemmas, Part-of-speech (POS) tags.
4. Identify and display different POS categories such as: Nouns, Verbs, Adjectives, Adverbs. 5. Count and display the frequency of the major POS categories present in the reviews. **Named Entity Recognition (NER)**

6. Apply Named Entity Recognition (NER) using spaCy to identify entities in the reviews. Extract and display entities such as: PERSON, ORG, DATE, MONEY, QUANTITY, PRODUCT, where recognized by the selected spaCy model. Display the extracted information in a structured DataFrame.

Information Extraction

7. Create a function that accepts a customer review as input and returns: Tokens, Lemmas, POS tags, Named entities, Entity labels.
8. Test the function using at least three new customer reviews.
9. Display the extracted information in a structured format.

[9]
```
# Name: Nithin S.
# Roll No: 241801191

import pandas as pd
import spacy
from collections import Counter
```

[10]
```
# Name: Nithin S.
# Roll No: 241801191

%pip install pandas spacy
!python -m spacy download en_core_web_sm
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: spacy in /usr/local/lib/python3.13/dist-packages (3.8.16)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.4)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.13)
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.3.13)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.5.4)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.0)
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.3.3)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (0.27.2)
Requirement already satisfied: click<9.0.0,>=8.2.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.5.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.13.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.13/dist-packages (from spacy) (80.10.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (26.3)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (2.46.5)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->spacy) (0.4.4)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.20)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2026.7.22)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from thinc<8.4.0,>=8.3.12->spacy) (1.3.3)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (1.5.4)
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 22.0 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

[12]
```
# Name: Nithin S.
# Roll No: 241801191

df = pd.read_csv("Amazon_Fine_Food_Reviews_Practice.csv")

print(df.head())
print(df.tail())
print("Shape:", df.shape)
print(df.dtypes)
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
print(df.describe(include="all"))
```

```
1     Electronics    Meera  2025-01-02
2     Electronics    Priya  2025-01-03
3           Books    Meera  2025-01-04
4           Books    Rahul  2025-01-05
     ReviewID                                               Text  Rating  \
115       116  The Philips mug is useful. I bought 3 units fo...       5
116       117  The Sony backpack is affordable. I bought 4 un...       2
117       118  The Sony charger is useful. I bought 4 units f...       2
118       119  The Target tea is useful. I bought 3 units for...       5
119       120  The Cetaphil water bottle is useful. I bought ...       5

    ProductCategory Reviewer  ReviewDate
115            Food    Priya  2025-04-26
116          Beauty    Aarav  2025-04-27
117           Books    Aarav  2025-04-28
118            Home    Rahul  2025-04-29
119           Books    Priya  2025-04-30
Shape: (120, 6)
ReviewID           int64
Text              object
Rating             int64
ProductCategory   object
Reviewer          object
ReviewDate        object
dtype: object
ReviewID          0
Text              0
Rating            0
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 22.0 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

[12] ✓ 0s

```
# Name: Nithin S.
# Roll No: 241801191

df = pd.read_csv("Amazon_Fine_Food_Reviews_Practice.csv")

print(df.head())
print(df.tail())
print("Shape:", df.shape)
print(df.dtypes)
print(df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
print(df.describe(include="all"))
```

```
1     Electronics    Meera  2025-01-02
2     Electronics    Priya  2025-01-03
3           Books    Meera  2025-01-04
4           Books    Rahul  2025-01-05
     ReviewID                                               Text  Rating  \
115       116  The Philips mug is useful. I bought 3 units fo...       5
116       117  The Sony backpack is affordable. I bought 4 un...       2
117       118  The Sony charger is useful. I bought 4 units f...       2
118       119  The Target tea is useful. I bought 3 units for...       5
119       120  The Cetaphil water bottle is useful. I bought ...       5

    ProductCategory Reviewer  ReviewDate
115            Food    Priya  2025-04-26
116          Beauty    Aarav  2025-04-27
117           Books    Aarav  2025-04-28
118            Home    Rahul  2025-04-29
119           Books    Priya  2025-04-30
Shape: (120, 6)
ReviewID           int64
Text              object
Rating             int64
ProductCategory   object
Reviewer          object
ReviewDate        object
dtype: object
ReviewID          0
Text              0
Rating            0
```

[13] ✓ 0s

```
# Name: Nithin S.
# Roll No: 241801191

df = df[["Text"]].dropna()
df["Text"] = df["Text"].astype(str).str.strip()
df = df[df["Text"] != ""]
df = df.drop_duplicates(subset="Text")
df = df.head(10000).reset_index(drop=True)

print("Cleaned shape:", df.shape)
print(df.head())
```

```
Cleaned shape: (120, 1)
                                                Text
0  I love this organic green tea from Amazon. The...
1  Nature's Bakery makes excellent fig bars; I bo...
2  The headphones from Sony are comfortable, but ...
3  Dr. Emily Carter recommended this vitamin supp...
4  This coffee maker by Philips is worth $89 and ...
```

[14] ✓ 0s

```
# Name: Nithin S.
# Roll No: 241801191

nlp = spacy.load("en_core_web_sm")

print("spaCy model loaded successfully")
```

```
spaCy model loaded successfully
```

[15] ✓ 0s

```
# Name: Nithin S.
# Roll No: 241801191

for review in df["Text"].head(5):
    doc = nlp(review)

    print("\nReview:", review)
    print("Tokens:", [token.text for token in doc])
    print("Lemmas:", [token.lemma_ for token in doc])
    print("POS Tags:", [(token.text, token.pos_) for token in doc])
```

```
Review: I love this organic green tea from Amazon. The package arrived on March 5, 2024, and cost $12.99.
Tokens: ['I', 'love', 'this', 'organic', 'green', 'tea', 'from', 'Amazon', '.', 'The', 'package', 'arrived', 'on', 'March', '5', ',', '2024', ',', 'and', 'cost', '$', '12.99', '.']
Lemmas: ['I', 'love', 'this', 'organic', 'green', 'tea', 'from', 'Amazon', '.', 'the', 'package', 'arrive', 'on', 'March', '5', ',', '2024', ',', 'and', 'cost', '$', '12.99', '.']
POS Tags: [('I', 'PRON'), ('love', 'VERB'), ('this', 'DET'), ('organic', 'ADJ'), ('green', 'ADJ'), ('tea', 'NOUN'), ('from', 'ADP'), ('Amazon', 'PROPN'), ('.', 'PUNCT'), ('The', 'DET'), ('package',

Review: Nature's Bakery makes excellent fig bars; I bought 3 boxes yesterday.
Tokens: ['Nature', "'s", 'Bakery', 'makes', 'excellent', 'fig', 'bars', ';', 'I', 'bought', '3', 'boxes', 'yesterday', '.']
```

[16]

```
# Name: Nithin S.
# Roll No: 241801191

counts = Counter()

for review in df["Text"]:
    doc = nlp(review)
    counts.update(token.pos_ for token in doc)

doc = nlp(df["Text"][0])

print("Nouns:", [t.text for t in doc if t.pos_ in ["NOUN", "PROPN"]])
print("Verbs:", [t.text for t in doc if t.pos_ == "VERB"])
print("Adjectives:", [t.text for t in doc if t.pos_ == "ADJ"])
print("Adverbs:", [t.text for t in doc if t.pos_ == "ADV"])

for pos in ["NOUN", "PROPN", "VERB", "AUX", "ADJ", "ADV"]:
    print(pos, ":", counts[pos])
```

```
Nouns: ['tea', 'Amazon', 'package', 'March']
Verbs: ['love', 'arrived', 'cost']
Adjectives: ['organic', 'green']
Adverbs: []
NOUN : 640
PROPN : 122
VERB : 316
AUX : 97
ADJ : 115
ADV : 5
```

[17]

```
# Name: Nithin S.
# Roll No: 241801191

entities = []

for i, review in enumerate(df["Text"], start=1):
    doc = nlp(review)

    for ent in doc.ents:
        entities.append({
            "ReviewID": i,
            "Review": review,
            "Entity": ent.text,
            "Entity Label": ent.label_
        })

entities_df = pd.DataFrame(entities)
```

```
    ReviewID                                             Review  \
0          1  I love this organic green tea from Amazon. The...
1          1  I love this organic green tea from Amazon. The...
2          1  I love this organic green tea from Amazon. The...
3          2  Nature's Bakery makes excellent fig bars; I bo...
4          2  Nature's Bakery makes excellent fig bars; I bo...
5          2  Nature's Bakery makes excellent fig bars; I bo...
6          3  The headphones from Sony are comfortable, but ...
7          3  The headphones from Sony are comfortable, but ...
8          4  Dr. Emily Carter recommended this vitamin supp...
9          4  Dr. Emily Carter recommended this vitamin supp...
10         5  This coffee maker by Philips is worth $89 and ...
11         5  This coffee maker by Philips is worth $89 and ...
12         5  This coffee maker by Philips is worth $89 and ...
13         6  The chocolate tastes delicious and the 500 gra...
14         7  Apple customer service replaced my damaged dev...
15         7  Apple customer service replaced my damaged dev...
16         8  I ordered two bottles of olive oil from Califo...
17         8  I ordered two bottles of olive oil from Califo...
18         9  The book by John Smith explains machine learni...
19        10  These running shoes cost 75 dollars and were d...

          Entity Entity Label
0         Amazon          ORG
1   March 5, 2024         DATE
2           12.99        MONEY
3             fig          ORG
4               3     CARDINAL
5       yesterday         DATE
6            Sony          ORG
7    only 4 hours         TIME
8    Emily Carter       PERSON
9            2023         DATE
10       Philips          ORG
11            89        MONEY
12      two days          DATE
13      500 gram     QUANTITY
14         Apple          ORG
15    January 12          DATE
16           two     CARDINAL
17    California          GPE
18    John Smith       PERSON
19    75 dollars        MONEY
```

[18]

```
# Name: Nithin S.
# Roll No: 241801191

def extract_information(review):
    doc = nlp(review)

    return {
        "Tokens": [t.text for t in doc],
```

[18]

```
# Name: Nithin S.
# Roll No: 241801191

def extract_information(review):
    doc = nlp(review)

    return {
        "Tokens": [t.text for t in doc],
        "Lemmas": [t.lemma_ for t in doc],
        "POS Tags": [t.pos_ for t in doc],
        "Named Entities": [e.text for e in doc.ents],
        "Entity Labels": [e.label_ for e in doc.ents]
    }

reviews = [
    "Ravi bought a Sony speaker for $45 in Chennai.",
    "Priya purchased two bottles of shampoo yesterday.",
    "The Samsung phone arrived on January 12, 2025."
]

for review in reviews:
    print("\nReview:", review)
    print(extract_information(review))
```

```
Review: Ravi bought a Sony speaker for $45 in Chennai.
{'Tokens': ['Ravi', 'bought', 'a', 'Sony', 'speaker', 'for', '$', '45', 'in', 'Chennai', '.'], 'Lemmas': ['ravi', 'buy', 'a', 'Sony', 'speaker', 'for', '$', '45', 'in', 'Chennai', '.'], 'POS Tags': [

Review: Priya purchased two bottles of shampoo yesterday.
{'Tokens': ['Priya', 'purchased', 'two', 'bottles', 'of', 'shampoo', 'yesterday', '.'], 'Lemmas': ['Priya', 'purchase', 'two', 'bottle', 'of', 'shampoo', 'yesterday', '.'], 'POS Tags': ['PROPN', 'VE

Review: The Samsung phone arrived on January 12, 2025.
{'Tokens': ['The', 'Samsung', 'phone', 'arrived', 'on', 'January', '12', ',', '2025', '.'], 'Lemmas': ['the', 'Samsung', 'phone', 'arrive', 'on', 'January', '12', ',', '2025', '.'], 'POS Tags': ['DE
```

[19]

```
# Name: Nithin S.
# Roll No: 241801191

entities_df.to_csv("ex5_extracted_entities.csv", index=False)

results = []

for i, review in enumerate(df["Text"], start=1):
    info = extract_information(review)

    results.append({
        "ReviewID": i,
```

[19]

```
# Name: Nithin S.
# Roll No: 241801191

entities_df.to_csv("ex5_extracted_entities.csv", index=False)

results = []

for i, review in enumerate(df["Text"], start=1):
    info = extract_information(review)

    results.append({
        "ReviewID": i,
        "Review": review,
        "Tokens": ", ".join(info["Tokens"]),
        "Lemmas": ", ".join(info["Lemmas"]),
        "POS Tags": ", ".join(info["POS Tags"]),
        "Named Entities": ", ".join(info["Named Entities"]),
        "Entity Labels": ", ".join(info["Entity Labels"])
    })

result_df = pd.DataFrame(results)
result_df.to_csv("ex5_review_information.csv", index=False)

print("Files saved successfully!")
print(result_df.head())
```

```
Files saved successfully!
   ReviewID                                             Review  \
0         1  I love this organic green tea from Amazon. The...
1         2  Nature's Bakery makes excellent fig bars; I bo...
2         3  The headphones from Sony are comfortable, but ...
3         4  Dr. Emily Carter recommended this vitamin supp...
4         5  This coffee maker by Philips is worth $89 and ...

                                              Tokens  \
0  I, love, this, organic, green, tea, from, Amaz...
1  Nature, 's, Bakery, makes, excellent, fig, bar...
2  The, headphones, from, Sony, are, comfortable,...
3  Dr., Emily, Carter, recommended, this, vitamin...
4  This, coffee, maker, by, Philips, is, worth, $...

                                              Lemmas  \
0  I, love, this, organic, green, tea, from, Amaz...
1  nature, 's, Bakery, make, excellent, fig, bar,...
2  the, headphone, from, Sony, be, comfortable, ,...
3  Dr., Emily, Carter, recommend, this, vitamin, ...
4  this, coffee, maker, by, Philips, be, worth, $...

                                           POS Taes  \
```

Principles of Data Science

Ex8: Interactive Dashboards with Tableau

Learning Objective:

To understand and implement interactive data visualization and dashboard design using Tableau with the Sample Superstore dataset. The objective is to create meaningful visualizations such as bar charts, line charts, and scatter plots to analyze sales, profit, and regional performance. It also aims to develop skills in applying filters, dashboard actions, and interactive features to explore data effectively and support data-driven decision-making.

Built-in Functions:

SUM() – Calculates the total sales and profit.

YEAR() – Extracts the year from the order date.

MONTH() – Extracts the month from the order date.

AVG() – Calculates the average sales or profit.

COUNT() – Counts the number of records.

IF() – Applies conditional logic.

FILTER() – Filters data based on specified conditions.

SORT() – Arranges data in ascending or descending order. MAX() – Returns the maximum value of sales or profit.

MIN() – Returns the minimum value of sales or profit.

COUNTD() – Counts distinct values in a field.

ROUND() – Rounds a number to a specified number of decimal places.

DATEDIFF() – Calculates the difference between two dates.

DATEPART() – Extracts a specific part of a date.

ATTR() – Returns a single value for a field when possible.

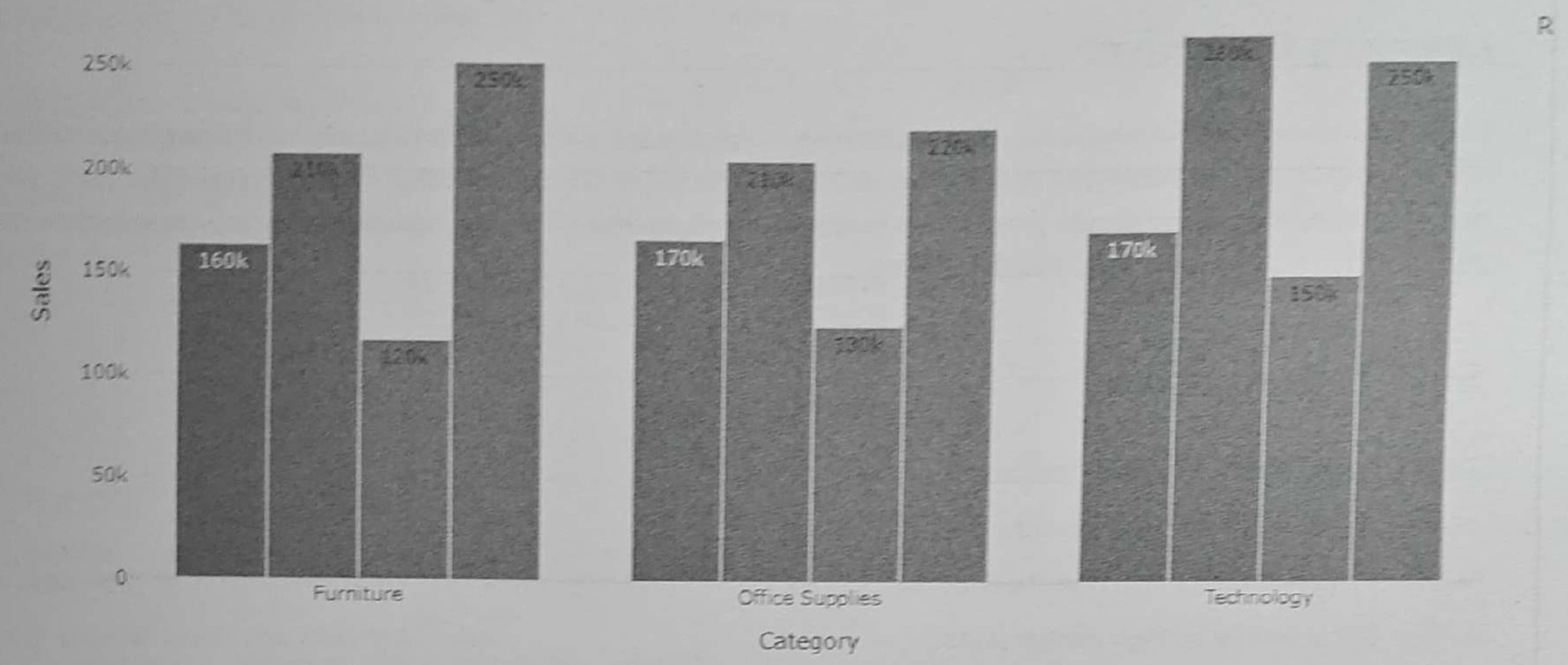
ZN() – Replaces null values with zero.

```
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from dash import Dash, dcc, html, Input, Output
df = pd.read_csv("/content/SampleSuperstore.csv", encoding="latin1")
df.columns = [c.replace('_', ' ').replace('-', ' ').title() for c in df.columns]
date_cols = [c for c in df.columns if "Date" in c]
if date_cols:
    target_col = date_cols[0]
    df[target_col] = pd.to_datetime(df[target_col])
    df["Year-Month"] = df[target_col].dt.to_period("M").astype(str)
else:
    df["Year-Month"] = "Unknown"
print("Dataset successfully loaded! Current columns:")
print(list(df.columns))
```

```
Dataset successfully loaded! Current columns:
['Ship Mode', 'Segment', 'Country', 'City', 'State', 'Postal Code', 'Region', 'Category', 'Sub Category', 'Sales', 'Quantity
```

```
bar_data = df.groupby(
    ["Category", "Region"], as_index=False
)["Sales"].sum()
fig = px.bar(
    bar_data,
    x="Category",
    y="Sales",
    color="Region",
    barmode="group",
    text_auto=".2s",
    title="Sales by Category"
)
```

```
fig.update_layout(template="plotly_white")
fig.show()
```



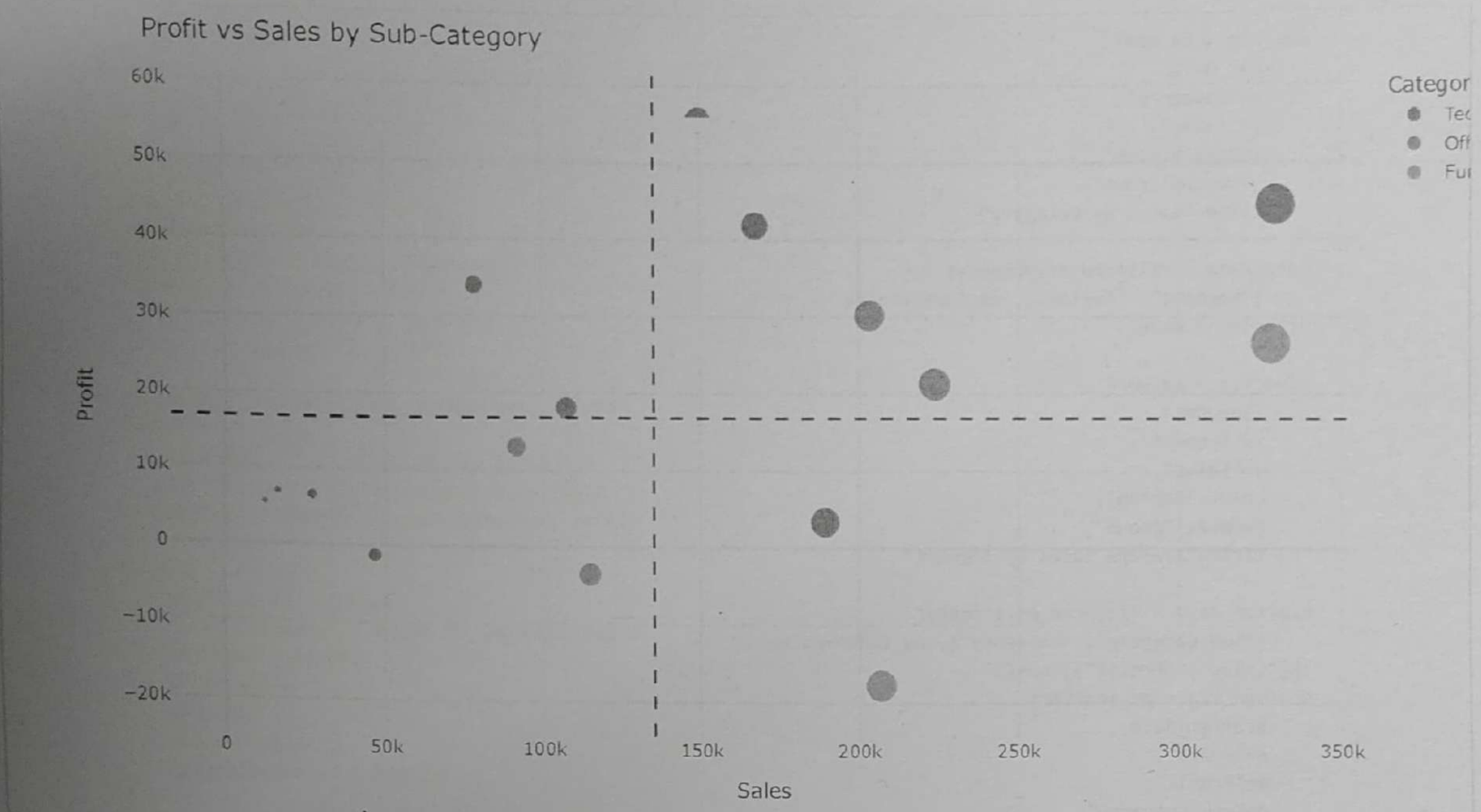

```
trend_data = df.groupby(['Segment', 'Region'], as_index=False)[['Sales', 'Profit']].mean()
fig = px.bar(
    trend_data,
    x="Segment",
    y="Profit",
    color="Region",
    barmode="group",
    title="Average Profit by Segment & Region"
)

fig.update_layout(
    template="plotly_white",
    xaxis_title="Segment",
    yaxis_title="Average Profit"
)
fig.show()
```




```
dfg = df.groupby(['Sub Category', 'Category'], as_index=False)[['Sales', 'Profit']].sum()
fig = px.scatter(dfg, x='Sales', y='Profit', color='Category', size='Sales', hover_name='Sub Category', title='Profit vs S
fig.add_hline(y=dfg['Profit'].mean(), line_dash='dash')
fig.add_vline(x=dfg['Sales'].mean(), line_dash='dash')
```

```
fig.update_layout(template='plotly_white', margin=dict(l=10, r=10, t=30, b=10))
fig.show()
```




```
app = Dash(__name__)
regions = sorted(df["Region"].dropna().unique())
categories = sorted(df["Category"].dropna().unique())
segments = sorted(df["Segment"].dropna().unique())
app.layout = html.Div([
    html.H1(
        "Sales and Profit Dashboard",
        style={"textAlign": "center", "fontFamily": "sans-serif"}
    ),
    html.Label("Select Region"),
    dcc.Dropdown(
        id="region-filter",
        options=[{"label": r, "value": r} for r in regions],
        value=regions,
        multi=True
    ),
    html.Label("Select Category"),
    dcc.Dropdown(
        id="category-filter",
        options=[{"label": c, "value": c} for c in categories],
        value=categories,
        multi=True
    ),
    html.Label("Select Segment"),
    dcc.Dropdown(
        id="segment-filter",
        options=[{"label": s, "value": s} for s in segments],
        value=segments,
        multi=True
    ),
    dcc.Graph(id="bar-chart"),
    dcc.Graph(id="line-chart"),
    dcc.Graph(id="scatter-chart")
])
@app.callback(
    Output("bar-chart", "figure"),
    Output("line-chart", "figure"),
    Output("scatter-chart", "figure"),
    Input("region-filter", "value"),
    Input("category-filter", "value"),
    Input("segment-filter", "value")
)
def update_dashboard(selected_regions, selected_categories, selected_segments):
    filtered_df = df[
        df["Region"].isin(selected_regions or []) &
        df["Category"].isin(selected_categories or []) &
        df["Segment"].isin(selected_segments or [])
    ].copy()
```

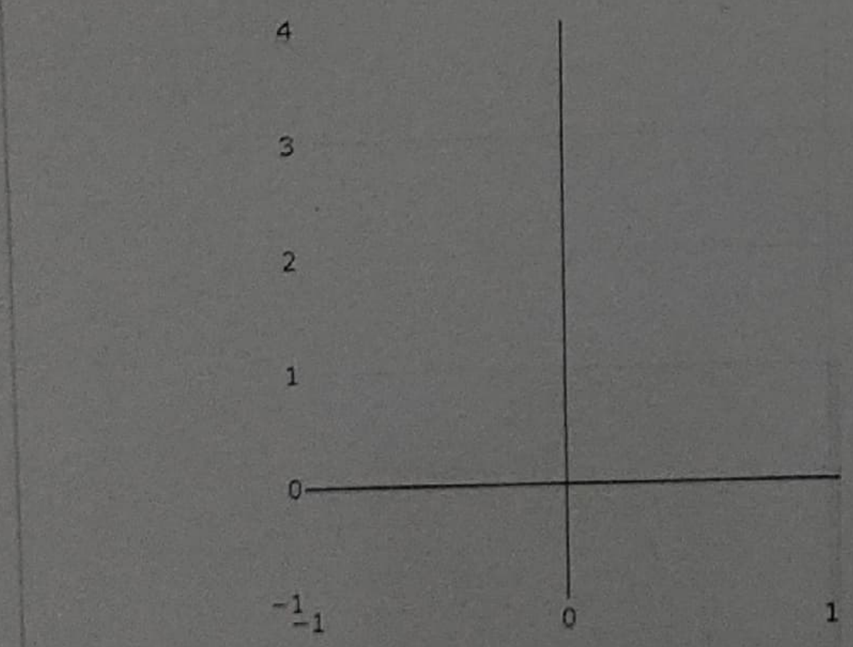
```
    ["Category", "Region"], as_index=False
)["Sales"].sum()

bar_fig = px.bar(
    bar_data,
    x="Category",
    y="Sales",
    color="Region",
    barmode="group",
    title="Sales by Category"
)
line_data = filtered_df.groupby(
    ["Segment", "Region"], as_index=False
)["Sales"].mean()

line_fig = px.bar(
    line_data,
    x="Segment",
    y="Sales",
    color="Region",
    barmode="group",
    title="Average Sales by Segment"
)
scatter_data = filtered_df.groupby(
    ["Sub Category", "Category"], as_index=False
)[["Sales", "Profit"]].sum()
scatter_fig = px.scatter(
    scatter_data,
    x="Sales",
    y="Profit",
    color="Category",
    size="Sales",
    hover_name="Sub Category",
    title="Profit vs Sales by Sub-Category"
)
    for fig in [bar_fig, line_fig, scatter_fig]:
        fig.update_layout(template="plotly_white")
    return bar_fig, line_fig, scatter_fig
if __name__ == "__main__":
    print("Starting Dash server...")
    app.run(debug=True, port=8050)
```

```
Starting Dash server...
```

Sales and Profit Dashboard



Errors

ID not found in layout	8:25:53 AM
ID not found in layout	8:25:53 AM
ID not found in layout	8:25:53 AM

Plotly Cloud | Errors 3 | Callbacks | v4.4.1 | Server

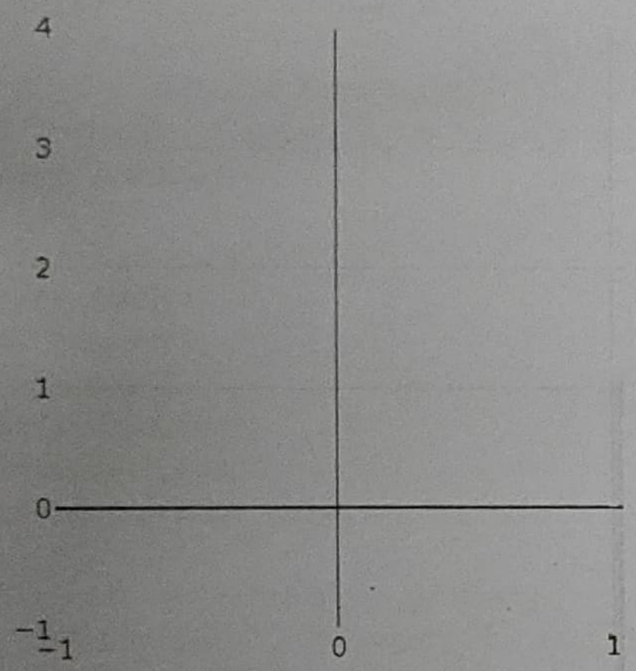
```
app = Dash(__name__)
```

```
    html.Div([
        dcc.Graph(id="bar-chart"),
        dcc.Graph(id="line-chart"),
        dcc.Graph(id="scatter-chart")
    ])
])
```

```
@app.callback(
    Output("bar-chart", "figure"),
    Output("line-chart", "figure"),
    Output("scatter-chart", "figure"),
    Input("region-filter", "value"),
    Input("category-filter", "value"),
    Input("year-filter", "value")
)
def update_dashboard(regions, categories, years):
    filtered_df = df[
        df["Region"].isin(regions or []) &
        df["Category"].isin(categories or []) &
        df["Order Date"].dt.year.isin(years or [])
    ]
    bar_fig = px.bar(
        filtered_df.groupby(
            ["Category", "Region"], as_index=False
        )["Sales"].sum(),
        x="Category",
        y="Sales",
        color="Region",
        title="Sales by Category"
    )
    line_fig = px.line(
        filtered_df.groupby(
            ["Year-Month", "Region"], as_index=False
        )["Sales"].sum(),
        x="Year-Month",
        y="Sales",
        color="Region",
        title="Sales Trend Over Time"
    )
    scatter_fig = px.scatter(
        filtered_df.groupby(
            ["Sub-Category", "Category"], as_index=False
        )[["Sales", "Profit"]].sum(),
        x="Sales",
        y="Profit",
        color="Category",
        hover_name="Sub-Category",
        title="Profit vs Sales by Sub-Category"
    )
    return bar_fig, line_fig, scatter_fig
```

```
if __name__ == "__main__":
    app.run(debug=True, port=8050)
```

Sales and Profit Dashboard



Errors

ID not found in layout 8:27:42 AM

ID not found in layout 8:27:42 AM

ID not found in layout 8:27:42 AM

Plotly Cloud | Errors 3 | Callbacks | v4.4.1 | Server

Learning Outcome:

1.Understand how to load and explore datasets in Tableau.

2.Create bar charts, line charts, and scatter plots.

3.Design interactive dashboards using multiple worksheets.

4.Apply filters and dashboard actions for dynamic data exploration.

5.Analyze sales trends, regional performance, and profitability.